

POWER BI DAX

From Complete Beginner to Professional DAX Developer

A Practical, Step-by-Step Learning Manual

*Volume 1 — Foundations, the Data Model and DAX Language Fundamentals
(Modules 1 to 3), with a Preview Pack and Flashcards*

COURSE PHILOSOPHY. Do not learn DAX as a dictionary of functions. Learn how Power BI represents data, how calculations are stored or evaluated, how context changes results, and how business questions become reusable analytical calculations.

Contents

How to Use This Manual	5
Who this manual is for.....	5
What this course is really trying to teach you	5
The learning pattern used in every module.....	5
How to study each module.....	6
Course roadmap	6
The module map.....	8
The logical order of ideas	9
What you should be able to say after Modules 1 and 2.....	10
MODULE 1 — THE WORLD OF POWER BI AND DAX	11
Learning Objectives	11
What Is Data?	11
What Is Data Analysis?	11
What Is Business Intelligence?	12
What Is Power BI?.....	12
What Is a Semantic Model?	13
What Is DAX?	13
Why Do We Need DAX?.....	14
What Is a Calculation? (Three Levels of Question)	15
A Very Important Question: When Should a Calculation Happen?	15
Persistent / Stored Calculations	15
On-the-Fly / Dynamic Calculations	16
Two Simple Analogies	16
Important Terminology.....	17
Important Correction: Persistent Does Not Mean Permanent.....	17
Refresh-Time vs Query-Time	18
A Careful Word About "On-the-Fly"	18
What Lives Inside a Model? Source Columns, DAX Columns, DAX Tables and Measures	19
Why Do Different Calculation Types Exist?	19
The Three Major DAX Calculation Types	20
Calculated Tables: What They Are For.....	21
Stored vs Dynamic: The Comparison	21
Calculated Column vs Measure	21
The Four Homes of DAX.....	22
Why Were Measures Needed?.....	22
Why Measures Are So Important	23
A Measure Is a Business Definition.....	23
Implicit and Explicit Measures	24
Measures Can Be Reused: Measure Branching	24
How a Business Question Becomes DAX	25
The Three Layers of a Power BI Solution	26
DAX vs Excel Formulas	26
DAX vs SQL.....	26
DAX vs Power Query	27
Where Should the Work Be Done? The Upstream Rule	27
A Practical Example	28
When Should I Use a Calculated Column?.....	29
When Should I Use a Measure?.....	29
A Decision Guide: What Should I Create?.....	30

Wrong Approach vs Correct Approach	30
The Beginning of Filter Context	31
How Power BI "Sees" a Report	31
DAX Is Not Just a List of Functions	32
The DAX Learning Pyramid	32
DAX Vocabulary You Must Know Before Module 3	33
The Core Mental Model	34
After the Calculation: Helping the Reader See the Result	34
Module 1 Summary	35
Module 1 Knowledge Check	35
Answers and Explanations — Module 1	36
Module 1 Practice	36
Module 1 Foundation Checklist	37
MODULE 2 — UNDERSTANDING DATA AND THE SEMANTIC MODEL	38
Learning Objectives	38
What Is a Table?	38
What Is a Row?	38
What Is a Column?	39
What Is Grain?	39
Why Grain Matters	39
What Is a Field?	39
Data Types	40
What Is BLANK?	40
What Is a Unique Value?	40
Keys	41
Relationships	41
Cardinality and the One-to-Many Relationship	42
Fact Tables	42
Dimension Tables	42
Star Schema	43
Why Professional Models Prefer One Direction	43
Clean Data Comes First: Typical Power Query Steps	44
Why the Model Matters to DAX	44
How Filters Travel: Filter Propagation	45
How Power BI Evaluates a Visual	45
Active and Inactive Relationships	46
The Date Table	46
Business Definitions Come Before DAX	46
The Professional DAX Workflow	47
Measure Branching and Model Design	47
The Core Mental Model — Stored or Dynamic	48
Model Validation Checklist	48
Module 2 Knowledge Check	49
Answers — Module 2 Knowledge Check	49
Module 2 Practical Model Exercise	49
Answers — Module 2 Practical Model Exercise	50
Module 2 Mini-Project	51
Module 2 Professional Challenge	51
The Foundation Diagram	52
The First Major Lesson: Five Ideas to Remember	52
Final Foundation Checklist (Modules 1 and 2)	52
MODULE 3 — DAX LANGUAGE FUNDAMENTALS	52
Learning Objectives	54

What Is a DAX Expression?	54
A DAX Formula Has a Structure	55
The Left Side and Right Side	55
Functions	55
Arguments	56
Parentheses	56
Tables and Columns	56
Measures Are Referenced Differently	57
Constants and Literals	57
Text Values	57
Arithmetic Operators	58
Comparison Operators	58
Logical Operators	58
The DIVIDE Function	59
Operator Precedence	59
A Measure Can Use Another Measure	59
Columns vs Measures — Syntax Practice	59
A DAX Formula Is an Expression Tree	60
Variables — VAR and RETURN	60
Why Use Variables?	60
Comments in DAX	61
Professional Naming	61
Formatting DAX	61
Common Syntax Errors	62
Read the Error, Do Not Panic	62
The Three Questions to Ask When a Formula Fails	62
First Complete DAX Exercise	62
Practice: Read Before You Write	63
Module 3 Assessment	64
Answers — Module 3	64
Module 3 Professional Challenge	65
Module 3 Mental Model	65
Final Skills Checklist (Modules 1 to 3)	65
What Comes Next?	65
PREVIEW PACK — THE NEXT MODULES AT A GLANCE	67
Lens 1 and Lens 2: Row Context and Filter Context	67
Row Context Does Not Follow Relationships by Itself: RELATED and RELATEDTABLE	67
Iterators: Why SUMX Has an "X"	67
CALCULATE: The Function That Changes the Filter Context	69
Time Intelligence: Comparing With the Past	69
Visual Calculations: Calculating on What the Visual Already Shows	70
DAX Query View and Copilot	70
The Complete Anatomy of a Professional Power BI Workflow	70
Preview Pack Knowledge Check	71
Answers — Preview Pack	71
APPENDIX — QUICK-REVIEW FLASHCARDS	71

How to Use This Manual

Who this manual is for

This course assumes **you know nothing about DAX**. You do not need to know any of these things before you start:

- DAX
- Power BI
- data modelling
- SQL
- programming
- database design

We will build your knowledge slowly, one idea at a time. Every major idea is introduced in **simple language first**, and only then developed in a more technical way. If you can read a sales list and understand what it says, you can learn DAX from this manual.

What this course is really trying to teach you

The goal is **not** to make you memorise DAX functions. Functions are easy to look up. The goal is for you to understand six questions that sit behind every calculation you will ever write:

1. **What problem are we solving?**
2. **What calculation do we need?**
3. **Where should the calculation live?**
4. **When should it be calculated?**
5. **What data should it see?**
6. **Why does the result change?**

Once you understand those six questions, DAX becomes much easier. Keep coming back to them. When a formula confuses you later in the course, go back to these six questions and ask them one by one.

DO NOT MEMORISE EVERYTHING AT ONCE. At the beginning, focus only on the mental model: **Data → Model → Calculation → Context → Result**. The functions will be introduced gradually. You do not need to memorise every DAX function. You need to understand how the pieces fit together.

The learning pattern used in every module

Each major idea in this manual follows the same pattern, so you always know where you are:

1. **Simple definition** — what it is, in plain words.
2. **Why the concept exists** — the problem it was invented to solve.
3. **Real-world analogy** — something from everyday life that behaves the same way.
4. **How Power BI handles it** — what actually happens inside the tool.

5. **Basic example** — the smallest possible example.
6. **Business example** — how it looks in a real company.
7. **Common mistakes** — what beginners usually get wrong.
8. **Professional guidance** — what experienced developers do.
9. **Practice exercises** — questions for you to answer yourself.
10. **Answers and explanations** — check your own thinking.
11. **Mini-projects** — put several ideas together.

How to study each module

- Read a section slowly. Do not rush to the formulas.
- Say each definition out loud in your own words. If you cannot, read it again.
- Answer every knowledge-check question **before** you read the answers.
- When a table or diagram appears, redraw it on paper once. This helps your memory more than re-reading.
- Have Power BI Desktop open on your laptop when you reach examples with formulas. Type the formulas yourself instead of copying and pasting. Typing builds the habit.
- Do not skip the early modules. Many people struggle with advanced DAX because they tried to learn advanced formulas before understanding the model and context.

Course roadmap

The full course is arranged in ten parts. **This volume covers Part 1, Part 2 and the start of Part 3 (Modules 1 to 3).** The later parts are shown so that you can see where the journey is going. This volume also ends with a **Preview Pack** (the engine ideas behind later modules, in simple words) and an **Appendix of quick-review flashcards**.

Part	Focus	What You Will Learn
Part 1	Foundations	Power BI, DAX, data, semantic models, stored vs dynamic calculations
Part 2	DAX Fundamentals	Syntax, expressions, data types, operators and functions
Part 3	Calculation Types	Calculated columns, measures and calculated tables
Part 4	How DAX Thinks	Row context, filter context and context transition
Part 5	Core DAX	CALCULATE, FILTER, iterators and table functions
Part 6	Time and Business Analysis	Dates, time intelligence, percentages, ranking, sales and customers
Part 7	Advanced DAX	Variables, virtual tables, advanced context and relationship patterns
Part 8	Professional DAX	Debugging, optimisation, VertiPaq, DAX Studio and security
Part 9	Projects	Sales, retail, banking, manufacturing, HR, healthcare and e-commerce

Part	Focus	What You Will Learn
Part 10	Reference	Function dictionary, patterns, cheat sheets and capstone

What is inside each stage

Stage 1 — Understanding Power BI

- Introduction to Power BI
- Introduction to DAX
- Understanding data
- Tables, rows and columns
- Relationships
- Data models
- Persistent/stored vs on-the-fly calculations

Stage 2 — Understanding DAX

- DAX syntax
- Data types
- Operators
- Functions
- Expressions
- Calculated columns
- Measures
- Calculated tables

Stage 3 — Understanding How DAX Thinks

- Row context
- Filter context
- Context transition
- CALCULATE
- FILTER
- Iterators

Stage 4 — Analytical DAX

- Date calculations
- Time intelligence
- Percentages
- Ranking
- Customer analysis
- Sales analysis
- Budget analysis
- Financial analysis
- Inventory analysis

Stage 5 — Advanced DAX

- Variables
- Virtual tables
- Advanced table functions
- Advanced CALCULATE patterns
- Dynamic DAX
- Advanced context
- Relationship functions

Stage 6 — Professional DAX

- Debugging
- Performance optimisation
- Formula Engine
- Storage Engine
- VertiPaq
- DAX Studio
- Performance Analyzer
- Professional DAX development
- Row-Level Security
- DAX queries

Stage 7 — Real Business Projects

- Sales dashboard
- Retail dashboard
- Banking analysis
- Manufacturing analysis
- HR analytics
- Healthcare analytics
- E-commerce analytics
- Executive performance dashboard
- Capstone project

Stage 8 — Reference

- DAX function dictionary
- DAX cheat sheets
- Professional DAX patterns
- Becoming a professional DAX analyst

The module map

The order in which ideas are taught matters. This is the exact order used in the course. Notice that the concept of **context** is met gently early on and only studied formally after you know what calculated columns, measures and calculated tables are.

Module	Title	Status
1	The World of Power BI and DAX	In this volume
2	Understanding Data and the Semantic Model	In this volume
3	DAX Language Fundamentals	In this volume
Preview	Preview Pack: the next modules at a glance	In this volume
4	Calculated Columns (and row context)	Next volume
5	Measures (and filter context)	Next volume
6	Calculated Tables	Next volume
7	DAX Data Types and Operators	Next volume
8	Aggregation Functions	Next volume
9	Logical Functions	Next volume
10	Text Functions	Next volume
11	Row Context	Next volume
12	Filter Context	Next volume
13	Context Transition	Next volume
14	CALCULATE	Next volume
15 onward	Iterators, time intelligence, advanced DAX, performance, professional development, projects	Later volumes

The logical order of ideas

The whole course follows this single chain. Each block needs the one above it.

DATA
↓
TABLES
↓
ROWS & COLUMNS
↓
GRAIN
↓
DATA TYPES
↓
RELATIONSHIPS
↓
DATA MODEL
↓
BUSINESS QUESTIONS
↓
CALCULATIONS
↓
STORED VS DYNAMIC
↓
CALCULATED COLUMNS
↓
MEASURES
↓
CALCULATED TABLES
↓
DAX SYNTAX
↓
EXPRESSIONS & FUNCTIONS
↓
ROW CONTEXT
↓
FILTER CONTEXT
↓
CONTEXT TRANSITION
↓
CALCULATE
↓
ITERATORS
↓
TIME INTELLIGENCE
↓
ADVANCED DAX
↓
PERFORMANCE
↓
PROFESSIONAL DEVELOPMENT

What you should be able to say after Modules 1 and 2

By the end of the first two modules you should be able to say the following paragraph **in your own words**. If you can, Module 3 and everything after it will be much easier.

THE PARAGRAPH TO OWN. “I have data. Power BI organises that data into a semantic model. Some calculations can be materialised and stored in the model, while measures are calculation definitions evaluated according to the current report/query context. Calculated columns answer row-level questions; measures answer analytical questions across the model. Relationships determine how the data interacts. DAX is the language I use to express those business calculations.”

MODULE 1 — THE WORLD OF POWER BI AND DAX

Learning Objectives

By the end of this module you will be able to:

- Understand data, data analysis and Business Intelligence.
- Understand what Power BI is and what role DAX plays inside it.
- Understand the idea of a **semantic model**.
- Understand what a calculation is, and why different kinds of calculations exist.
- Tell apart **stored/materialised** calculations from **query-time/on-the-fly** calculations.
- Understand calculated columns, measures and calculated tables.
- Understand why measures are central to interactive reporting.
- Begin thinking in terms of **business questions** rather than isolated formulas.

What Is Data?

Before learning DAX, we need to understand the thing DAX works with.

Data is information collected about business activities, people, products, transactions or other events. Data is simply facts that someone wrote down or a system recorded.

Imagine a shop records every sale:

Date	Product	Quantity	Unit Price	Customer	Region
Jan 1	Laptop	2	500,000	John	South
Jan 2	Mouse	5	10,000	Mary	West
Jan 3	Laptop	1	500,000	Peter	South

This is data. It records **sales events**. Each row has a meaning (one recorded sale), and each column describes one attribute of that row:

- **Date** tells us when the sale happened.
- **Product** tells us what was sold.
- **Quantity** tells us how many were sold.
- **Unit Price** tells us the price of one item.
- **Customer** tells us who bought it.
- **Region** tells us where the sale belongs.

Notice that the table above does not say “how much did the shop sell in total?”. The facts are there, but the answer is not. Turning facts into answers is what the rest of this course is about.

What Is Data Analysis?

Data analysis means **examining data to answer useful questions**. For example:

- How much did we sell?
- Which product sold the most?
- Which region generated the most revenue?
- What was our profit?
- Did sales increase this month?
- Which customers buy the most?
- How did sales change from last year?
- What was the growth in each region?

CORE IDEA. Raw data does not automatically become insight. We need definitions, calculations, context and interpretation. That is where Power BI and DAX become important.

What Is Business Intelligence?

Business Intelligence, commonly called **BI**, is the process of turning business data into useful information that supports analysis and decision-making. In other words, BI helps a manager decide what to do next using facts instead of guesses.

A simple way to think about it:

Raw Data → Analysis → Information → Business Insight

Here is the same idea with real numbers:

- **Raw data:** 1,000 individual sales transactions.
- **Analysis:** calculate total sales.
- **Information:** total sales = ₦250 million.
- **Insight:** sales increased by 18% compared with the previous period.

Each step is more useful to a manager than the one before it. A manager cannot make a decision by reading 1,000 rows, but “sales are up 18%” can change a decision immediately.

What Is Power BI?

Microsoft **Power BI** is Microsoft’s business intelligence and analytics platform. It can connect to data, transform data, build a semantic model, create calculations, produce reports and support interactive analysis. Power BI allows us to:

- import data
- clean data
- model data
- create relationships
- calculate values
- create reports
- create dashboards
- visualise information
- share analytical results

Power BI is therefore **much more than a charting tool**. Charts are only the last step. A simplified Power BI workflow is:

```
Get Data
↓
Transform Data
↓
Build Data Model
↓
Create Calculations
↓
Build Visuals
↓
Analyse Results
```

In plain words, each step means:

1. **Get Data** — connect to the place where your data lives (an Excel file, a CSV file, a database, a website).
2. **Transform Data** — clean it up: fix wrong types, remove errors, fill gaps, rename columns.
3. **Build Data Model** — organise the tables and connect them with relationships.
4. **Create Calculations** — write the business logic (this is where DAX lives).
5. **Build Visuals** — place charts, tables and cards on a report page.
6. **Analyse Results** — click, filter and explore the report to find answers.

What Is a Semantic Model?

You will hear the term **semantic model** very often in professional Power BI work, so we introduce it early.

A **semantic model** is the organised analytical structure Power BI uses to represent business data. It contains tables, columns, relationships, measures and other model metadata (“metadata” simply means information *about* the data, such as column names, data types and how tables connect).

Think of it as **the organised analytical world in which DAX operates**.

```
Tables + Columns + Relationships + Measures + Model Metadata = Semantic Model
```

An everyday picture: a library is not just a pile of books. It has shelves, sections, a catalogue and rules for how things are connected. Because of that organisation you can ask “show me all the science books written after 2010” and get an answer quickly. A semantic model is the library organisation for your data. In older material you may see the words “data model” or “Power BI model”. In this course we use *data model*, *model* and *semantic model* to mean the same thing, and we prefer **semantic model** because that is the term used in professional Power BI work.

What Is DAX?

DAX means **Data Analysis Expressions**. It is the formula language used to create analytical calculations in Power BI and in other Microsoft tabular technologies.

For example:

```
Total Sales =  
SUM(Sales[Sales Amount])
```

This defines a **measure** called Total Sales. In plain words it tells Power BI: *“Add together the values in the Sales Amount column.”*

Another example:

```
Total Quantity =  
SUM(Sales[Quantity])
```

This tells Power BI: *“Add together the values in the Quantity column.”*

DAX allows us to create analytical calculations such as:

- total sales
- profit
- profit margin
- average sales
- sales growth
- year-to-date sales
- previous-year sales
- percentage of total
- ranking
- customer lifetime value
- budget variance

Why Do We Need DAX?

Businesses rarely want only raw rows. They want **answers**, such as: What are total sales? What is profit? How did sales change from last year? What percentage of sales came from each region?

Imagine your company has **10 million** sales transactions. Your manager asks:

“What were sales for the South region in 2026?”

Then:

“What about January?”

Then:

“What about laptops only?”

Then:

“What about January laptop sales in the South?”

Then:

“What percentage of total company sales came from that region?”

Every question is slightly different. No one can write a separate formula for each possible question. The calculation must **respond to the question being asked**. This is one of the major reasons DAX is

powerful: DAX lets us define **business calculations** once, and Power BI evaluates them against whichever part of the data the report is asking about.

What Is a Calculation? (Three Levels of Question)

Before we compare calculation types, we need a very clear idea of what a calculation is and at what **level** a question is being asked. Ask yourself: *is my question about one row, about a group of rows, or about whatever the report is showing right now?*

Level	The question asks...	Example
Row level	What is true about this particular row ?	The value of one transaction: Quantity × Unit Price
Group level	What is true about a group of rows ?	Sales for the South region
Model / report level	What is true according to the current report context ?	Total Sales for whatever the user has selected in the report

```
Sales Amount =  
Sales[Quantity] * Sales[Unit Price]
```

The formula above is a **row-level** calculation: it gives every row its own answer. But:

```
Total Sales =  
SUM(Sales[Sales Amount])
```

is a **report-level** calculation: it gives one answer for whatever set of rows the report is currently looking at.

Once you see these levels, you can understand why a measure is not simply “another column”. They answer different levels of question. Keep this table in mind. It explains why DAX has different calculation types.

A Very Important Question: When Should a Calculation Happen?

This question is the bridge between beginner and professional DAX. Before we learn measures and calculated columns, we need to answer:

THE QUESTION. When should a calculation happen?

There are two important ideas:

1. **Persistent / Stored calculations.** The result is calculated and stored (materialised) in the model.
2. **On-the-fly / Dynamic calculations.** The calculation is evaluated when a report or query asks for the result.

These two ideas explain most of the difference between **calculated columns** and **measures**.

Persistent / Stored Calculations

The words you may meet for this idea are:

- persistent

- stored
- materialised
- precomputed

They all describe the same idea: the result is **calculated during model processing or refresh, and then stored in the model**. After that, it simply sits there like any other column of data.

Consider this formula:

```
Sales Amount =
Sales[Quantity] * Sales[Unit Price]
```

Suppose the table contains:

Quantity	Unit Price
2	500
3	100
5	50

The calculation produces:

Quantity	Unit Price	Sales Amount
2	500	1,000
3	100	300
5	50	250

Read the first row: $2 \times 500 = 1,000$. The result column can be **materialised** in the model: Power BI works it out once, then keeps the answers. This is the basic idea behind a **calculated column**.

On-the-Fly / Dynamic Calculations

Now imagine we create this measure:

```
Total Sales =
SUM(Sales[Sales Amount])
```

The measure does **not** create a new stored value for every row. Instead, Power BI evaluates the measure according to the **context of the report query**. For example:

- If the report simply shows **Total Sales**, the measure might return **£100 million**.
- If the user selects **Region = South**, the same measure may return **£35 million**.
- If the user then also selects **Product = Laptop**, the same measure may return **£12 million**.

The formula has not changed. The **context in which the formula is evaluated** has changed. This is the key to understanding measures.

Two Simple Analogies

The restaurant. Imagine a restaurant.

- **Persistent / stored:** you prepare 100 sandwiches in advance. They are already made and ready on the shelf.
- **On-the-fly:** a customer walks in and says, “Make me a chicken sandwich.” You prepare it based on that customer’s request.

The first approach is like a stored/materialised calculation. The second is like an on-the-fly calculation.

Photograph or live camera. A photograph is taken once and stays the same until you take a new one (stored). A live camera shows whatever is in front of it right now (dynamic). A calculated column is like a photograph of your data taken at refresh time. A measure is like a live view that changes with what you point it at.

This is only an analogy. The Power BI engine uses sophisticated storage and caching mechanisms. But these pictures give beginners the right mental model, and we will refine it later.

Important Terminology

Different books, instructors and courses use slightly different words for the same ideas. You may see any of these:

Term	Meaning
Persistent	Stored/materialised
Stored	The value exists in the model
Materialised	Calculated and stored
Precomputed	Calculated before query time
Dynamic	Depends on the current context
On-the-fly	Evaluated when requested
Query-time	Evaluated as part of a query

For this course we will use **Persistent/Stored Calculations** and **On-the-Fly/Dynamic Calculations**, because those names make the difference easy to understand. We will keep using this pair of ideas **throughout the whole course**, not just in this module.

TERMINOLOGY CARE. Do not memorise “persistent = calculated column” as an absolute rule. In this course, *persistent/stored calculations* means values that are materialised in the model, such as calculated columns and calculated tables. *Measures* are definitions whose results are evaluated at query time. The link is a useful guide, not a law of nature.

Important Correction: Persistent Does Not Mean Permanent

A calculated column may be stored in the model after a refresh. But that does not mean it can never change. If the underlying data changes and the model is **refreshed**, Power BI can recalculate the column.

Therefore:

PERSISTENT ≠ PERMANENT. Persistent means the result is materialised/stored as part of the processed model. It does not mean the value is frozen forever. A refresh can recalculate it.

Refresh-Time vs Query-Time

A learner will soon ask: “If a calculated column is stored, when does it change?” The answer is that stored values change at **refresh time**, while measure results change at **query time** (whenever the report asks a new question).

For stored/materialised calculations:

```
Source Data Changes
    ↓
Power BI Refresh
    ↓
Stored/Materialised Calculations Recomputed
    ↓
Updated Model
```

For measures:

```
User Selects "South"
    ↓
Report Query Changes
    ↓
Measure Evaluated in the New Context
    ↓
Context-Specific Result Appears
```

Or, written in the general form:

```
User / Visual / Query Context
    ↓
Measure Evaluated
    ↓
Context-Specific Result
```

Two clocks, two moments: **refresh time** (data is loaded and stored calculations are produced) and **query time** (someone looks at the report and measures are worked out).

A Careful Word About “On-the-Fly”

For learning purposes we call measures *on-the-fly/dynamic* calculations, because their results are evaluated according to the current query context. But this is a **teaching model**, not a literal engineering description.

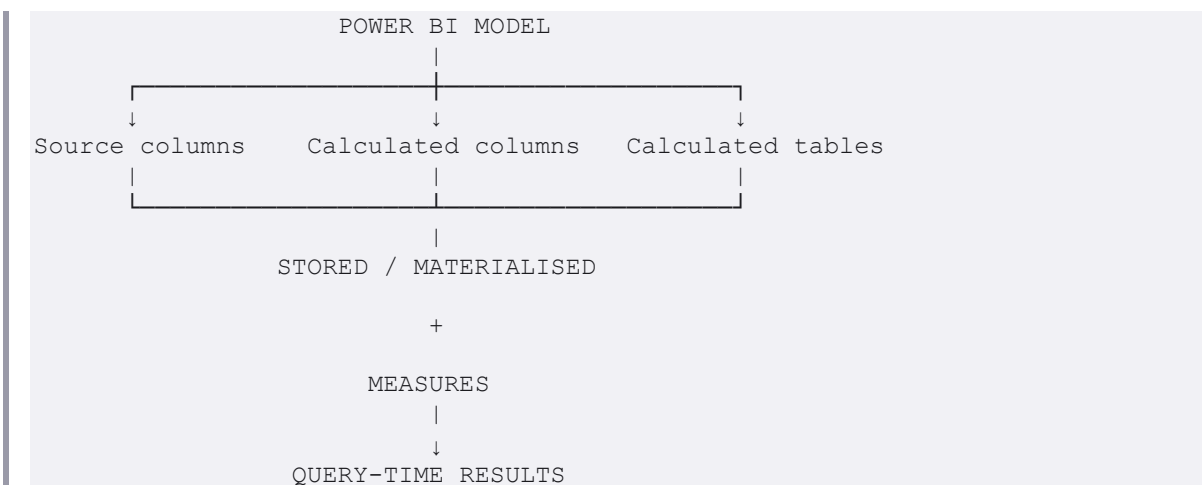
TECHNICAL NUANCE. Calling measures “on-the-fly” does not mean Power BI starts from zero and recalculates everything from scratch every time. The engine can cache results, optimise queries, use its Storage Engine and Formula Engine, and reuse work. The important distinction is that the measure result is **evaluated for the current query context**.

In the performance modules later in the course we will explain the Storage Engine and Formula Engine properly. For now, remember only this: *a measure result depends on context, and a stored column value depends on refresh.*

What Lives Inside a Model? Source Columns, DAX Columns, DAX Tables and Measures

It would be too narrow to think only “persistent means calculated column”. There are actually several things inside a model to tell apart:

- **Source/model columns:** data that was already loaded into Power BI from the outside (for example, the Quantity column that came from Excel).
- **Calculated columns:** DAX creates a new column, and the resulting values are materialised in the model.
- **Calculated tables:** DAX creates a whole table, and that table is materialised in the model.
- **Measures:** the measure *definition* is stored, but its analytical *result* is evaluated when queried.



This is a more accurate picture than “columns are stored and measures are not”. A measure’s *formula* is saved in the file, but its *answer* is created when the report asks.

Why Do Different Calculation Types Exist?

The reason is that **business questions exist at different levels**. Here is the story a beginner should understand.

First, we have data. Quantity, price, cost, date, customer, product.

Then we need row-level information. For example: “What is the value of this particular transaction?” That naturally leads us to **calculated columns**:

```
Sales Amount =  
Sales[Quantity] * Sales[Unit Price]
```

Then we need business-level questions. For example: What are total sales? What are sales for Lagos? What are sales for January? What are sales for the selected product? What is sales growth? These are **not properties of an individual row**. They are questions about *a set of data*. That naturally leads us to **measures**:

```
Total Sales =  
SUM(Sales[Sales Amount])
```

Sometimes we need a whole new table built from the model, which leads us to **calculated tables**.

Question Level	Typical Question	Natural Calculation
Row level	What is the value of this transaction?	Calculated column
Analytical / report level	What are total sales for the selected region?	Measure
Table level	Can I create a new analytical table from the model?	Calculated table

The Three Major DAX Calculation Types

Now we can introduce the three major DAX calculation types formally.

1. Calculated Columns

A calculated column creates a **value for each row** of a table. Conceptually it answers: *“What value should this row have?”*

```
Sales Amount =
Sales[Quantity] * Sales[Unit Price]
```

If Quantity is 2 and Unit Price is 500, the row’s Sales Amount is 1,000. Conceptually: **one row → one calculation result**. Calculated columns are generally materialised during model processing/refresh. They can be useful for row-level attributes, flags, categories and values needed as model fields.

2. Measures

A measure is a **calculation definition** that Power BI evaluates when it is requested.

```
Total Sales =
SUM(Sales[Sales Amount])
```

Conceptually: **question/context → calculation → result**. The result can change according to:

- filters
- slicers
- rows
- columns
- relationships
- report selections

Measures are therefore central to interactive Power BI analysis.

3. Calculated Tables

A calculated table creates a **table** using DAX.

```
ProductSummary =
SUMMARIZE(
    Sales,
    Sales[Product]
)
```

A calculated table is materialised as part of model processing/refresh.

Calculated Tables: What They Are For

Calculated tables have **specialised uses**. They are not a replacement for ordinary measures, and not a replacement for a well-designed source table or model table. Typical uses are a helper list of values, a date table built in DAX, or a summary table the model needs. We study them properly in their own module. For now, remember only that a calculated table is *a table created by DAX and stored in the model*.

Stored vs Dynamic: The Comparison

Feature	Stored / Materialised	Dynamic / Query-Time
Typical DAX examples	Calculated columns, calculated tables	Measures
When values are produced	During model processing/refresh	When queried/evaluated
Primary question	What value belongs in the model?	What is the result in this context?
Typical scope	Rows or tables	Report/query context
Responds to slicers by changing stored values?	No	Yes, through measure evaluation
Main consideration	Model storage and processing	Query evaluation and context

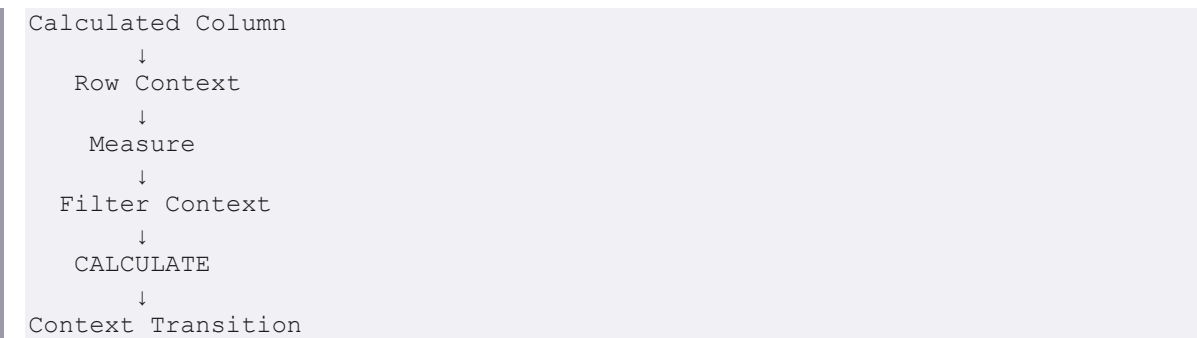
Calculated Column vs Measure

This distinction is one of the most important things a beginner must understand. Read this table twice.

Feature	Calculated Column	Measure
Produces	A value for each row	A result for a query/context
Materialised	Yes, after processing	The measure result is evaluated at query time
Row-by-row	Yes	Not inherently
Changes with slicers	Its stored row values do not	Yes
Uses row context naturally	Yes	Measures begin without an inherent row context
Can be used as a field	Yes	Yes, as a measure
Typical purpose	Create row-level attributes/calculations	Analyse and aggregate data
Example	Quantity × Price	Total Sales
Storage cost	Increases model size	Saves only the definition
Evaluated	At refresh	At query time

Feature	Calculated Column	Measure
Reacts to slicers	Not by itself (stored values stay)	Yes
Best use	Slicing, filtering, categories (for example Age Band)	Dynamic aggregations (for example Profit Margin)

Looking ahead, this gives us the logical path through the later modules:



A calculated column is naturally linked with **row context** (“what is the current row?”). A measure is naturally linked with **filter context** (“which rows are currently included?”). CALCULATE can modify filter context and, in the right circumstances, perform **context transition**. Do not worry about these words yet. They are the road ahead.

The Four Homes of DAX

Understanding DAX means knowing two things about every calculation: **where it reads its data from**, and **where its result lives**. Put those two questions on a map and you get four “homes” for DAX calculations.

	Output: stored in the model	Output: shown in a visual (on the fly)
Input: the physical data model	Calculated columns and calculated tables. Read the model and store the result back in the model (data to data).	DAX measures. Read the model and produce a result for a visual (data to visual).
Input: the data already shown in a visual	The impossible zone. A visual’s results cannot be written back into the stored model.	Visual calculations. Read the visual’s own data and produce a result in that visual (visual to visual).

INSIGHT. To understand any DAX calculation, ask: where does it draw its source data from, and where does its result live?

Visual calculations are a newer feature that you will meet in the preview chapter near the end of this volume.

Why Were Measures Needed?

This is an important conceptual question. Suppose you have **10 million sales rows** and you want *Total Sales*. You could calculate a value for every row and then add those values together. But many analytical questions are not simply row-level calculations. Consider:

- What percentage of total company sales came from each region?
- What were sales last year?
- What is sales growth?
- What is the average sales value for customers who purchased this month?
- What was the result after removing the product filter?

These questions depend heavily on **filter context and the report query**. A measure provides a **reusable business calculation** that can be evaluated against the current analytical context. That is why measures are so important in Power BI.

Why Measures Are So Important

Many business questions are not properties of individual rows. They are questions about groups of rows under a particular analytical context.

Business Question	Why a Measure Fits
What are total sales?	It summarises a set of sales rows.
What are South-region sales?	The same calculation can respond to Region = South.
What is profit margin?	It can use reusable business measures.
What is sales growth?	It can compare context-specific periods.
What percentage of total sales is this category?	It depends on the current report context.

A Measure Is a Business Definition

Consider:

```
Total Sales =
SUM(Sales[Sales Amount])
```

Do not think of this merely as a formula. Think of it as a **business definition**: *“Whenever I ask for Total Sales, calculate the sum of Sales Amount according to the current context.”*

A measure is better understood as a **named DAX expression representing a business calculation that Power BI evaluates within the current filter/query context**. A professional analyst thinks of a measure as a *reusable definition of a business concept*.

Break the example down:

Part	Role
Total Sales	The name of the measure
=	Assignment (connects the name to the expression)
SUM	The function
Sales[Sales Amount]	The column reference

Part	Role
The whole expression	The business calculation definition

Now the same measure can appear in:

- a card
- a table
- a matrix
- a chart
- a tooltip
- a KPI
- another measure

This is one of the foundations of professional DAX development. The diagram to keep in mind from now on is:

```

Visual
  ↓
Query
  ↓
Filter Context
  ↓
Measure
  ↓
Result

```

Implicit and Explicit Measures

When you drag a numeric column such as `Sales Amount` straight into a visual, Power BI quietly adds it up for you and shows a small sigma (Σ) beside the field. That automatic total is called an **implicit measure**: Power BI created it for you without you writing any DAX.

When you write the formula yourself, it is an **explicit measure**:

```

Total Sales =
SUM(Sales[Sales Amount])

```

PROFESSIONAL HABIT. Do not rely on implicit measures. Always write explicit measures.

Why?

- **Control.** An implicit measure picks a default (usually Sum). That is wrong for columns such as Unit Price or Year, where adding up makes no sense. You choose the correct calculation yourself.
- **Reuse.** You cannot refer to an implicit measure by name inside other measures. An explicit measure such as `[Total Sales]` can feed `[Profit]`, `[Profit Margin]` and many more.
- **One place to change.** If the business changes the definition, you fix it once.
- **Clarity.** An explicit measure has a business name, a format and a place in the model that everyone can see.

Measures Can Be Reused: Measure Branching

Suppose we create:

```
Total Sales =
SUM(Sales[Sales Amount])

Total Cost =
SUM(Sales[Cost Amount])

Profit =
[Total Sales] - [Total Cost]

Profit Margin =
DIVIDE([Profit], [Total Sales])
```

Notice something important: we are **building calculations from other calculations**. This is called **measure branching**: simple base measures are reused to build more advanced business metrics. The structure becomes:

```
Base Measures
    ↓
Intermediate Measures
    ↓
Business Metrics
    ↓
Advanced KPIs
```

If the business later changes how “Total Sales” is defined, you change it in **one place** and every measure built on it updates. This is a professional DAX development pattern.

How a Business Question Becomes DAX

Example business request:

MANAGER’S QUESTION. How much profit did we make?

Do not start with a function. Start with the meaning. Suppose the manager first asks: *“How much money did the company make from sales?”* We first clarify the business definition. If “sales” means revenue:

```
Total Sales =
SUM(Sales[Sales Amount])
```

Now the manager asks: *“How much profit did we make?”* We need the business definition: **Profit = Sales – Cost**. Therefore:

```
Total Cost =
SUM(Sales[Cost Amount])

Profit =
[Total Sales] - [Total Cost]
```

Then: *“What percentage of sales was profit?”*

```
Profit Margin =
DIVIDE(
    [Profit],
    [Total Sales]
)
```

The professional sequence is:

Business Question → Business Definition → Required Data → Model → Calculation Type → DAX → Validation

This is how professional DAX should be learned. Not “*I know 500 functions*”, but “*I know how to translate a business question into a calculation.*”

The Three Layers of a Power BI Solution

A useful mental model is to see a Power BI solution as three layers.

Layer 1 — Data. The actual information: Sales, Customers, Products, Dates, Stores, Employees.

Layer 2 — Model. How the data is organised and connected. For example, Customers connect to Sales, Products connect to Sales, and Dates connect to Sales.

Layer 3 — Calculations. The business logic: Total Sales, Profit, Profit Margin, Sales Growth, YTD Sales.

```
Layer 3: CALCULATIONS    (Total Sales, Profit, Growth, YTD)
      ↑
Layer 2: MODEL           (tables and relationships)
      ↑
Layer 1: DATA           (Sales, Customers, Products, Dates)
```

A weak model can make even excellent DAX difficult. A good model makes DAX much easier.

DAX vs Excel Formulas

DAX looks somewhat like Excel formulas. For example:

- Excel: `=SUM(A1:A100)`
- DAX: `Total Sales = SUM(Sales[Sales Amount])`

But DAX is designed for a **data model and analytical context**, not individual spreadsheet cells. Excel formulas often make a beginner think in terms of cells and ranges. DAX operates within a model.

Excel Mental Model	DAX Mental Model
Cell → formula → result	Model → context → expression → result
Often range-oriented	Table/model-oriented
Worksheet-centric	Semantic-model-centric

Put simply: in Excel you point at cells (`A1:A100`); in DAX you point at columns (`Sales[Sales Amount]`) and let the current filters decide which rows count. This difference becomes extremely important later.

DAX vs SQL

SQL is primarily used to **retrieve and manipulate data** in relational databases. Example:

```
SELECT SUM(SalesAmount)
FROM Sales;
```

DAX is primarily used to **define analytical calculations** inside tabular semantic models such as Power BI models. The languages can solve related problems, but they are **not interchangeable**. A professional analyst may use:

- SQL to obtain and prepare data
- Power Query to transform data
- Power BI modelling to structure data
- DAX to create analytical calculations

DAX vs Power Query

This is another major beginner question. **Power Query** is primarily used for importing, cleaning, transforming, combining and reshaping data. **DAX** is primarily used for analytical calculations: measures, calculated columns, calculated tables and business metrics.

Power Query	DAX
Connect and ingest data	Analyse data in the model
Clean and transform data	Create analytical calculations
Merge/append/reshape data	Create measures and DAX model calculations
Primarily data preparation	Primarily semantic-model analysis

BEGINNER RULE. Prepare and shape the data with Power Query where appropriate. Define reusable analytical logic and analyse the model with DAX. There are exceptions and advanced scenarios, but this is a useful starting principle.

Where Should the Work Be Done? The Upstream Rule

Power BI is a pipeline. Data moves from left to right through five stages.

```
1 EXTRACT      (source data)
   ↓
2 TRANSFORM    (Power Query)
   ↓
3 MODEL        (storage and relationships)
   ↓
4 CALCULATE     (DAX engine)
   ↓
5 VISUALISE     (report layer)
```

The same business result can often be produced at several stages. The professional rule is:

THE UPSTREAM RULE. A calculation done further to the **left (upstream)** improves performance and can be reused by more reports. Pushing work to the right makes the report do more work every time someone opens or clicks it.

This ladder shows where each tool sits, from most efficient for data preparation to least:

Level	Tool	What it is best for	Trade-off
1 (most upstream)	SQL or Python at the database	Preparing data once for everyone. Maximum reuse across projects.	Needs database access and skills.
2	Power Query	Cleaning and shaping data at load time. Data arrives already prepared.	Works on one table at a time unless you merge.
3	DAX calculated columns	Calculations that need to see the whole model (for example using <code>RELATED</code>). Calculated after load.	Takes up memory and refresh time.
4 (most downstream)	DAX measures	Dynamic analysis that must react to clicks, such as profit margin.	Not meant for static data preparation: results are produced when visuals ask for them.

The rule in one paragraph. If a calculation can be done at the database, do it there. If it must be done in Power BI, use Power Query. Use a DAX calculated column only when the calculation really needs to see the relationships in the model. Use a DAX measure for anything that must change with the report.

A word you will hear: **query folding**. It means Power Query translates your cleaning steps into a query that the source database runs for you, so the heavy work happens upstream instead of inside Power BI.

A NOTE ABOUT EARLIER EXAMPLES. In this manual we created `Sales Amount` as a calculated column to teach the idea of a row-level calculation. In a real project, if `Quantity` and `Unit Price` are in the same source table, creating that column upstream (in SQL or Power Query) is usually better.

A Practical Example

Imagine this raw data:

Product	Quantity	Price	Cost
Laptop	2	500	350
Mouse	5	20	10
Keyboard	3	50	25

We might create two calculated columns (row-level values):

```
Sales Amount =
Sales[Quantity] * Sales[Price]

Cost Amount =
Sales[Quantity] * Sales[Cost]
```

The results row by row: Laptop sales $2 \times 500 = 1,000$ and cost $2 \times 350 = 700$; Mouse sales $5 \times 20 = 100$ and cost $5 \times 10 = 50$; Keyboard sales $3 \times 50 = 150$ and cost $3 \times 25 = 75$.

Then measures (report-level answers):

```
Total Sales =  
SUM(Sales[Sales Amount])  
  
Total Cost =  
SUM(Sales[Cost Amount])  
  
Profit =  
[Total Sales] - [Total Cost]  
  
Profit Margin =  
DIVIDE([Profit], [Total Sales])
```

With these three rows: Total Sales = 1,250, Total Cost = 825, Profit = 425, and Profit Margin = $425 \div 1,250 = 0.34$, or 34%. Notice the architecture:

```
RAW DATA  
↓  
Calculated Columns  
↓  
Base Measures  
↓  
Business Measures  
↓  
KPIs
```

When Should I Use a Calculated Column?

A calculated column may be appropriate when you need a value that **belongs to each individual row**. Examples:

```
Full Name =  
Customers[First Name] & " " & Customers[Last Name]  
  
Sales Amount =  
Sales[Quantity] * Sales[Unit Price]  
  
Age =  
YEAR(TODAY()) - Customers[Birth Year]
```

Calculated columns can also be useful for:

- categorisation
- row-level flags
- grouping
- keys
- labels
- row-level business logic

But **do not automatically create a calculated column just because you can**. Every calculated column takes up space in the model and adds work at refresh time. Model size and performance matter.

When Should I Use a Measure?

Use a measure when **the answer needs to respond to report context**. Examples:

```
Total Sales =
SUM(Sales[Sales Amount])

Average Sales =
AVERAGE(Sales[Sales Amount])

Profit =
[Total Sales] - [Total Cost]

Profit Margin =
DIVIDE([Profit], [Total Sales])
```

Measures are usually the primary tool for analytical reporting.

A Decision Guide: What Should I Create?

Follow the arrows, answering each question honestly.

```
I need a calculation.
↓
Does the value belong to each individual row?
↓ YES
Do I need that row-level value stored as a model field?
↓ YES
Consider a CALCULATED COLUMN.

I need a calculation.
↓
Should the answer respond to report filters and current context?
↓ YES
Consider a MEASURE.

I need a new DAX-created table.
↓
Consider a CALCULATED TABLE.
```

Wrong Approach vs Correct Approach

Professional learning improves dramatically when you see mistakes next to the correct way. Here are three.

Mistake 1: Storing the same total on every row

A beginner may think: *“I need total sales, so I should create a calculated column.”* That is usually the **wrong mental model**. You generally do not need a row-by-row column to store the same overall total on every row. Instead:

```
Total Sales =
SUM(Sales[Sales Amount])
```

creates a **measure** that responds to report context.

Requirement	Beginner approach	Better approach
Show total sales	Calculated column containing the same total repeatedly	A measure: Total Sales = SUM(Sales[Sales Amount])

Mistake 2: Thinking a measure calculates only once

A beginner may also think: “A measure is just a formula that calculates once.” No. A measure is a calculation definition **evaluated in the context in which it is requested**. The same measure can produce:

Region	Total Sales
North	₦20M
South	₦35M
East	₦18M
West	₦27M

The same measure is being used. The context changes. Therefore the result changes.

Mistake 3: Dividing row-level columns for a margin

Requirement: calculate profit margin. A poor approach would be to write `Sales[Profit] / Sales[Sales]` in a place where row-level logic does not make sense for a whole group of rows (the division of each row would not give the margin of the group). The better analytical design is:

```
Profit Margin =  
DIVIDE(  
    [Profit],  
    [Total Sales]  
)
```

This teaches **measure thinking** rather than formula copying: add up the profit, add up the sales, *then* divide.

The Beginning of Filter Context

We now reach one of the most important words in DAX: **CONTEXT**. You will eventually learn three related ideas: **Row Context**, **Filter Context** and **Context Transition**. They explain why DAX behaves differently from ordinary spreadsheet formulas. For now, remember only:

REMEMBER. A measure is evaluated in a context.

Suppose a report contains a Region field and a Total Sales measure. When the visual evaluates the South row, the measure is evaluated under a context corresponding to South.

```
Visual shows Region = South  
    ↓  
Relevant filter context  
    ↓  
[Total Sales]  
    ↓  
South sales result
```

The measure name `Total Sales` could mean *all sales* when nothing is selected. Put it in a visual showing `Region = South` and it means *South sales*. Add `Product = Laptop` and `Year = 2026` and it means *South + Laptop + 2026 sales*.

You have just met the beginning of **filter context**. We will formally study it later. For now remember: a measure can return different results because the context in which it is evaluated changes.

How Power BI “Sees” a Report

Suppose the visual is:

Region	Total Sales
South	₦35M
West	₦27M
North	₦20M

Conceptually, Power BI does the following for each row of the visual:

```
Power BI sees "South"
    ↓
Creates a filter for South
    ↓
Evaluates [Total Sales]
    ↓
Only relevant Sales rows remain in context
    ↓
SUM(Sales[Sales Amount])
    ↓
₦35M
```

Then for the next row:

```
Power BI sees "West"
    ↓
Different context
    ↓
Same [Total Sales]
    ↓
₦27M
```

That is the beginning of understanding DAX properly.

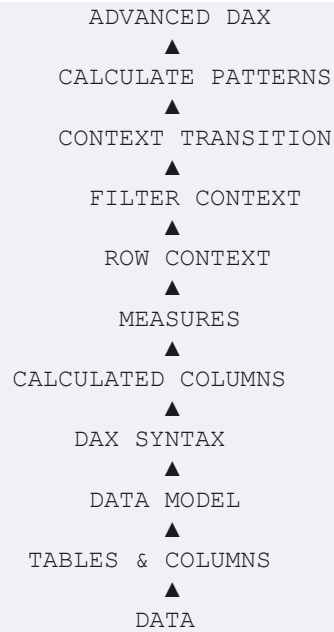
DAX Is Not Just a List of Functions

Knowing SUM, FILTER, CALCULATE, SUMX and hundreds of other functions does **not** automatically mean you understand DAX. A learner can memorise SUM, AVERAGE, COUNT, FILTER, CALCULATE, VALUES, ALL and SUMX and still not understand DAX.

PROFESSIONAL MINDSET. DAX is about expressions, tables, relationships, context and business logic. Functions are tools inside that larger system.

The DAX Learning Pyramid

Think of your learning like a pyramid. Each level stands on the one below it.



Do not skip the bottom. Many people struggle with advanced DAX because they tried to learn advanced formulas before understanding the model and context.

DAX Vocabulary You Must Know Before Module 3

A small vocabulary will prevent confusion later.

Term	Meaning
Expression	A piece of DAX that produces a result
Function	A built-in operation such as SUM
Argument	A value supplied to a function
Column reference	A reference to a model column
Measure	A named analytical calculation
Calculated column	A DAX-created row-level column
Calculated table	A DAX-created table
Context	The environment in which DAX is evaluated
Row context	The current row
Filter context	The active filters affecting a calculation
Relationship	A connection between tables
Model	A structured collection of tables and relationships
Semantic model	The analytical model Power BI uses
Implicit measure	An automatic total Power BI creates when a numeric column is dropped in a visual

Term	Meaning
Explicit measure	A measure you write yourself in DAX
Iterator	A DAX function ending in X (such as SUMX) that works row by row
Query folding	Power Query steps being run by the source database
VertiPaq	The in-memory storage engine behind Power BI models
Visual calculation	DAX calculated on the data inside a single visual

The Core Mental Model

```

DATA
↓
TABLES
↓
ROWS + COLUMNS
↓
GRAIN
↓
RELATIONSHIPS
↓
SEMANTIC MODEL
↓
BUSINESS QUESTION
↓
CALCULATION TYPE
↓
DAX
↓
CONTEXT
↓
RESULT

```

After the Calculation: Helping the Reader See the Result

The last pipeline stage is the report itself. A page full of raw numbers tires the reader. Three simple features help the eye understand quickly.

- **Hierarchies and drill-down.** A hierarchy is an ordered set of fields, such as Year → Quarter → Month. In a visual you can start at Year and drill down to see Quarters, then Months, without building separate charts.
- **Grouping and bins.** Grouping puts values into logical buckets. For example, grouping ages into 20s, 30s and 40s makes a chart readable instead of showing hundreds of single ages.
- **Conditional formatting.** Data bars, colours and icons let the eye see scale instantly. A long bar is large; a short bar is small. No need to read each number.

INSIGHT. Raw numbers exhaust the reader. Use data bars, sensible bins and drill-downs so the eye can judge scale and category without conscious reading. (Menu names for these features can vary slightly between Power BI versions.)

Module 1 Summary

You should now understand each of the following:

- **Data** — information collected about business activities.
- **Data Analysis** — using data to answer questions.
- **Business Intelligence** — turning business data into useful information for decisions.
- **Power BI** — a platform for data preparation, modelling, analysis and visualisation.
- **Semantic Model** — the organised analytical structure of tables, relationships and measures.
- **DAX** — Data Analysis Expressions, the formula language used for analytical calculations in Power BI.
- **Persistent/Stored Calculation** — a calculation whose result is materialised/stored in the model after processing.
- **On-the-Fly/Dynamic Calculation** — a calculation evaluated in response to a query/report context.
- **Calculated Column** — a DAX column that produces a value for each row and is materialised during model processing.
- **Measure** — a reusable calculation definition evaluated according to the current context.
- **Calculated Table** — a DAX-created table materialised during model processing.

Module 1 Knowledge Check

Answer these before moving forward. Write your answers first, then compare them with the answers below.

1. What does DAX stand for?
2. What is Power BI?
3. What is a semantic model?
4. What is the difference between a stored/materialised calculation and a query-time measure?
5. Why is persistent not the same as permanent?
6. Why is a calculated column naturally associated with row-level logic?
7. Why is a measure useful for interactive reporting?
8. Why can the same measure return different results?
9. What is the difference between a calculated column and a calculated table?
10. Why should you define the business question before writing DAX?
11. What does *persistent/stored* mean?
12. What does *on-the-fly/dynamic* mean?
13. What is the difference between a calculated column and a measure?
14. Which is more naturally suited to: “Calculate sales for whatever region the user selects.” (A) Calculated column (B) Measure
15. Which is more naturally suited to: “Calculate Quantity × Unit Price for every sale.” (A) Calculated column (B) Measure

Answers and Explanations — Module 1

1. **Data Analysis Expressions.**
2. A business intelligence and analytics platform used to connect, transform, model, analyse and visualise data.
3. The organised analytical model containing tables, relationships, measures and other model metadata.
4. Stored/materialised values are produced as part of model processing; measure results are evaluated for the current query context.
5. A refresh can recalculate stored DAX-created values when the underlying data changes.
6. A calculated column produces a value for each row and is materialised as a model column.
7. Measures can respond to filters, slicers, rows, columns and relationships.
8. The current query/filter context can change even though the measure definition stays the same.
9. A calculated column adds row-level values to a table; a calculated table creates a whole table.
10. Because the business definition determines what data, calculation type and logic are actually required.
11. The result is calculated during model processing/refresh and stored (materialised) in the model.
12. The calculation is evaluated when a report or query asks for the result, using the current context.
13. A calculated column produces a stored value for each row; a measure produces a result for a query/context. Columns are row-level; measures are report-level.
14. **B — Measure.** The answer must change with the user's selection, which is what measures do.
15. **A — Calculated column.** The value belongs to each individual sale row.

Module 1 Practice

Exercise A — Levels. For each question, say whether it is row level, group level or report level: (a) What is the value of sale number 1002? (b) What are total sales in the West region? (c) What are the sales for whatever the user has currently selected?

Answer: (a) row level; (b) group level; (c) report level.

Exercise B — Choose the calculation type. For each need, choose *calculated column*, *measure* or *calculated table*: (a) Mark each sale as Bulk if Quantity is 10 or more. (b) Show total profit for the selected year. (c) Create a small table listing each product once.

Answer: (a) calculated column (row-level flag); (b) measure (responds to the selected year); (c) calculated table.

Exercise C — Explain the paragraph. In your own words, write the paragraph from the start of this manual ("I have data. Power BI organises that data into a semantic model..."). Then check that you included: data, semantic model, stored vs query-time calculations, calculated columns, measures, relationships and DAX.

Module 1 Foundation Checklist

- I can explain what DAX is.
- I can explain what Power BI is.
- I understand the idea of a semantic model.
- I understand the difference between stored/materialised and query-time calculations.
- I know why calculated columns exist.
- I know why measures exist.
- I know what a calculated table is.
- I understand that a business definition comes before a DAX formula.

MODULE 2 — UNDERSTANDING DATA AND THE SEMANTIC MODEL

Before writing serious DAX, you must understand the data model. This is one of the biggest differences between a beginner and a professional. A beginner asks: “Which DAX function should I use?” A professional first asks: “How is the data structured?”

Learning Objectives

- Understand tables, rows, columns and fields.
- Understand grain and why it matters.
- Understand data types and blanks.
- Understand primary keys and foreign keys.
- Understand relationships and cardinality.
- Distinguish fact tables from dimension tables.
- Understand star schema.
- Understand date tables.
- Understand active and inactive relationships.
- Understand how filters travel through a model.
- Use model knowledge to choose appropriate DAX calculations.

What Is a Table?

A **table** is a structured collection of related information organised into **rows** and **columns**. If you have used Excel, a table is like a sheet of data with a header row, where every row is one record and every column is one kind of information.

Sale ID	Product	Quantity	Sales
1001	Laptop	2	1,000
1002	Mouse	5	100
1003	Keyboard	3	150

This is a table. In Power BI, a model usually has several tables, and each table has a name such as *Sales*, *Customers* or *Products*.

What Is a Row?

A **row** represents one record or event at the table’s defined grain. In a transaction table, a row may represent one sale or one line item.

Sale ID	Product	Quantity	Sales
1001	Laptop	2	1,000

This row might represent *one sales transaction*: one sale of 2 laptops for 1,000.

What Is a Column?

A **column** describes one attribute of the records. Examples: Product, Quantity, Sales, Date, Customer, Region. Every row has a value in each column, and every value in a column is the same kind of information (all dates, all names, all numbers).

What Is Grain?

Grain means **exactly what one row represents**. This is one of the most important questions to answer before writing DAX.

ASK THIS FIRST. What does one row of this table represent?

Examples of grain:

- One row = one sales transaction
- One row = one product per month
- One row = one employee
- One row = one inventory movement

Suppose one row equals one sales transaction. Then the grain is **one transaction per row**. But another table might say *one row = one product per month*. That is a completely different grain. Different grains lead to different calculations and different risks of duplication or double-counting.

Why Grain Matters

Look at this table:

Product	Date	Sales
Laptop	Jan	10,000
Laptop	Feb	15,000
Mouse	Jan	5,000

Here each row is **one product in one month**. The grain is *product-per-month*. If you do not notice this, you may make mistakes. For instance, if you tried to count “number of sales” by counting rows, you would get 3, but that is the number of *product-month combinations*, not the number of actual sales. And if you later connect this table to a customer table that has one row per customer, you could accidentally duplicate these amounts and overstate total sales.

DAX depends heavily on understanding what each row means. Before writing a formula, say the grain out loud: “*In this table, one row is _____.*”

What Is a Field?

A **field** is a data element that can be used in the model or report. Examples: Customer Name, Product, Sales Amount, Quantity, Date, Region. In Power BI, fields are listed in the Data pane and

can be dragged into visuals. A field can be a column or a measure. “Field” is the general word for anything you can drag onto a report.

Data Types

Every column has a **data type** that tells Power BI what kind of values it holds. Common data types:

Type	Examples
Text	Laptop, Nigeria, John
Whole Number	1, 25, 1000
Decimal Number	12.50, 99.95, 1000.75
Date	2026-01-01
Date/Time	2026-01-01 14:30
Boolean (True/False)	TRUE, FALSE

Understanding data types is important because **different DAX functions behave differently depending on the type of data**. You can add numbers but not names. A date stored as text cannot be used for year-to-date calculations. A number stored as text will not add up. Setting the correct type for every column is one of the cheapest ways to avoid errors later.

What Is BLANK?

DAX uses `BLANK()` to represent a blank result. A blank is **not automatically** the same thing as 0, and it is **not automatically** the same thing as an empty text string "".

```
BLANK()
```

Think of a blank as “nothing to show” or “no value”. A shop that sold zero laptops in June has sales of 0. A shop that has no record at all for June has a blank. They can look alike in a table but they mean different things. Professional DAX often requires **deliberate handling of blanks**: deciding whether “no data” should appear as nothing, as zero, or as a message.

What Is a Unique Value?

Suppose a column holds:

Region
South
South
North
West
South

The column has five rows, but only three **unique** (distinct) values: South, North and West. Understanding unique values is important for filtering, grouping, relationships, dimensions and analysis. For example, a list of unique regions is what a slicer shows you.

Keys

A **key** helps identify or connect records. Here is a **Customers** table:

CustomerID	Customer
101	John
102	Mary
103	Peter

And a **Sales** table:

SaleID	CustomerID	Sales
1	101	500
2	102	700
3	101	300

CustomerID connects the two tables. Sale 1 and sale 3 both belong to customer 101 (John). We do not repeat John's name in every sale. We store a short ID and look up the rest from the Customers table.

Primary key

A **primary key** uniquely identifies a row in its own table. For example, `CustomerID` can be the primary key of the Customers table if each customer has one unique ID. No two customers share the same ID, and none is left blank.

Foreign key

A **foreign key** contains values that refer to a key in another table. `Sales[CustomerID]` can act as a foreign key that points to `Customers[CustomerID]`. The word "foreign" just means "from another table".

```
Customers[CustomerID]  ← primary key (unique)
      ↑
Sales[CustomerID]      ← foreign key (repeats; points to the customer)
```

Relationships

A **relationship** connects tables so that the model understands how information in one table relates to information in another. It answers the question: *which customer belongs to each sale?*

```
Customers
|
| CustomerID
↓
Sales
```

In Power BI you create a relationship by matching a column in one table with a column in another (for example, `Customers[CustomerID]` with `Sales[CustomerID]`). Once connected, you can put `Customers[Region]` in a visual next to a Sales measure, and Power BI knows how to match each sale to its customer's region.

Cardinality and the One-to-Many Relationship

The word **cardinality** is used in two related ways, and you should know both:

1. **Cardinality of a column:** the number of unique values in a column. For example, a `CustomerID` column with values 101, 102, 103 has three unique values in this small example.
2. **Cardinality of a relationship:** how many rows on one side match how many rows on the other side.

Common relationship cardinalities are:

- **One-to-one** — each row on one side matches at most one row on the other side.
- **One-to-many** — one row on one side can match many rows on the other.
- **Many-to-one** — the same as one-to-many, read from the other direction.
- **Many-to-many** — many rows on each side can match many on the other.

A very common analytical pattern is **one customer to many sales**: one row on the Customers side can correspond to many rows on the Sales side. One customer can make many purchases.

```
One Customer
  ↓
Many Sales
Customers 1 ————— * Sales
```

The **1** means “one”. The ***** means “many”. The “one” side holds the unique list (Customers); the “many” side holds the repeating events (Sales). Cardinality becomes important when designing the model, because choosing the wrong one can give you wrong totals.

Fact Tables

A **fact table** usually records business events, transactions or measurable activities. Examples: Sales, Orders, Transactions, Production, Payments, Inventory movements. Fact tables tend to be long (many rows), and they hold numbers you want to add up.

```
FactSales
- SaleID
- DateID
- CustomerID
- ProductID
- Quantity
- Sales Amount
- Cost Amount
```

Dimension Tables

Dimension tables describe the entities (the “things”) involved in the business process: who, what, when, where. Dimension tables tend to be shorter and hold descriptions you want to group or filter by.

```
DimCustomer
- CustomerID
- Customer Name
- Gender
- City
- Region
- Customer Type
```

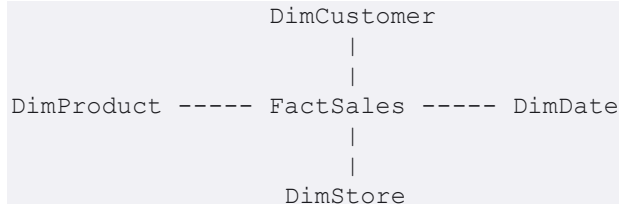
```
DimProduct
- ProductID
- Product Name
- Category
- Brand
```

```
DimDate
- DateID
- Date
- Year
- Month
- Quarter
- Week
```

A simple way to remember: **facts are what happened; dimensions describe who, what, when and where it happened.**

Star Schema

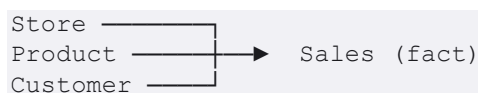
A common analytical design is the **star schema**: one central fact table surrounded by dimension tables. Drawn on paper it looks like a star.



Sales is the central fact table. The surrounding tables are dimensions. Star schemas are widely used because they give a clear structure for filtering, grouping and analytical calculations. Dimension tables connect to the fact table and normally not to each other. Understanding star schema is extremely important for professional Power BI work.

Why Professional Models Prefer One Direction

In a star schema, the arrows (filter direction) point **from each dimension table into the fact table**: from Store, Product and Customer into Sales. This is called a **unidirectional** (one-way) relationship.



It is tempting to make a relationship **bidirectional** (filters flow both ways) “just in case”. Resist that. When filters can travel both ways, there may be several paths between two tables, and Power BI can become unsure which path to use. That is called **ambiguity**, and it can produce unpredictable results and slow reports.

PROFESSIONAL RULE. Prefer strict one-to-many relationships with a single direction. This controls exactly how filter context flows. Use bidirectional filtering only for a specific, understood purpose.

Clean Data Comes First: Typical Power Query Steps

A beautiful model built on messy data gives wrong answers. Before modelling, the data is cleaned in Power Query. Three very common steps are:

Step	What it does	Why it matters
Promote headers	Turns the first row of data into the column names	Otherwise your column names are "Column1", "Column2"
Remove null rows	Deletes completely empty rows	Empty rows distort counts and averages
Unpivot columns	Turns many columns (Jan, Feb, Mar) into rows	Models and DAX work best with tall, narrow tables

Example of **unpivoting**. Before (wide):

Product	Jan	Feb	Mar
Laptop	10	12	9
Mouse	40	35	50

After (tall):

Product	Month	Quantity
Laptop	Jan	10
Laptop	Feb	12
Laptop	Mar	9
Mouse	Jan	40
Mouse	Feb	35
Mouse	Mar	50

The tall table can be related to a Date table and summed by month easily. The wide table cannot.

Why the Model Matters to DAX

Suppose you write:

```
Total Sales =  
SUM(Sales[Sales Amount])
```

Now you put **Product Category** from the Product table into a visual. Why does the Sales measure break down by Product Category, when the measure itself never mentions Category? **Because the tables are related.** A relationship tells Power BI how to carry a filter from one table to another.

```

DimProduct[Category]
    ↓
Relationship
    ↓
FactSales
    ↓
[Total Sales]

```

The data model determines how filters travel through the model. Therefore:

VERY IMPORTANT. DAX cannot be properly understood without understanding the data model. Understanding DAX without the model is like trying to understand traffic rules without understanding the roads.

How Filters Travel: Filter Propagation

Filter propagation is the name for how a filter chosen in one table reaches another table through a relationship. In a normal one-to-many relationship, the filter travels from the **“one” side to the “many” side**: a filter on Customers flows down to Sales, never the other way by default.

Example. Put `Customers[Region] = South` in a visual.

1. The filter starts on the Customers table: only customers in the South remain.
2. The relationship `Customers[CustomerID] → Sales[CustomerID]` carries the filter down.
3. Only sales rows belonging to those customers remain.
4. `[Total Sales]` adds up only those remaining rows.

```

Customers (Region = South)
    ↓ one-to-many
Sales rows of South customers only
    ↓
SUM(Sales[Sales Amount])

```

This is why `Total Sales` can respond to `Region` even though `Region` is stored in a different table and never appears in the formula.

How Power BI Evaluates a Visual

Suppose a visual contains `Region` and `Total Sales`. Conceptually, Power BI evaluates the measure once per region, each time under the context represented by that region.

```

Visual row: Region = South
    ↓
Relevant filter context
    ↓
[Total Sales]
    ↓
South sales result

Visual row: Region = West
    ↓
Different context
    ↓
[Total Sales]
    ↓
West sales result

```

This is the beginning of filter-context thinking. Later modules explain the exact mechanics.

Active and Inactive Relationships

A model can contain **more than one possible relationship** between tables. For example, Sales might contain *Order Date*, *Ship Date* and *Delivery Date*. The Date table could potentially relate to all three. However, normally **only one relationship between the same two tables can be active** for a filtering path. Active relationships are used automatically. Inactive relationships are shown as dashed lines in the Model view, and they are ignored unless a calculation asks for them.

In DAX, functions such as `USERELATIONSHIP()` can be used inside suitable measures to switch on an inactive relationship for that one calculation. This allows, for example, one measure for sales by Order Date and another for sales by Delivery Date.

WE WILL RETURN TO THIS. Relationship behaviour becomes especially important in advanced DAX and time-intelligence scenarios. For now, only know that active and inactive relationships exist.

The Date Table

A dedicated **Date table** provides a reliable calendar structure for time-based analysis. It has exactly one row for every day, with descriptive columns.

Date	Year	Month	Quarter	Month Number
2026-01-01	2026	January	Q1	1
2026-01-02	2026	January	Q1	1
2026-02-01	2026	February	Q1	2
2026-03-01	2026	March	Q1	3
...	...	...	...	...

A good Date table is what makes later calculations possible:

- Year-to-date sales
- Month-to-date sales
- Previous-year sales
- Year-over-year growth
- Rolling 12-month sales

The “Month Number” column is included so that months can be sorted in calendar order (January, February, March) instead of alphabetical order (April, August, December).

Business Definitions Come Before DAX

Suppose the manager says:

BUSINESS QUESTION. I want sales growth.

Do not immediately type DAX. First clarify:

- **What does “sales” mean?** Gross sales, net sales or revenue?
- **What does “growth” mean?** Growth compared with the previous month, previous quarter, previous year or budget?
- **Which date should drive the comparison?** Order date, ship date or delivery date?
- **Should the calculation respond to product, region and customer filters?**

Only after these questions are answered should the DAX design begin. The business definition comes first. Then the DAX.

The Professional DAX Workflow

```

BUSINESS QUESTION
  ↓
BUSINESS DEFINITION
  ↓
DATA REQUIRED
  ↓
DATA MODEL
  ↓
CALCULATION TYPE
  ↓
DAX
  ↓
VALIDATION
  ↓
REPORT
  
```

And here is the workflow that you should **avoid**:

```

QUESTION
  ↓
GOOGLE FUNCTION
  ↓
COPY FORMULA
  ↓
HOPE IT WORKS
  
```

Measure Branching and Model Design

Suppose you need Sales, Profit, Margin and Growth. Do not immediately create one enormous formula. Professional models build calculations **in layers** rather than putting everything into one expression.

```

Total Sales
  ↓
Total Cost
  ↓
Profit
  ↓
Profit Margin
  ↓
Advanced KPIs
  
```

```

Total Sales =
SUM(Sales[Sales Amount])

Total Cost =
SUM(Sales[Cost Amount])

Profit =
[Total Sales] - [Total Cost]

Profit Margin =
DIVIDE([Profit], [Total Sales])

```

This is cleaner, and it is easier to understand, test, debug, reuse and maintain.

The Core Mental Model — Stored or Dynamic

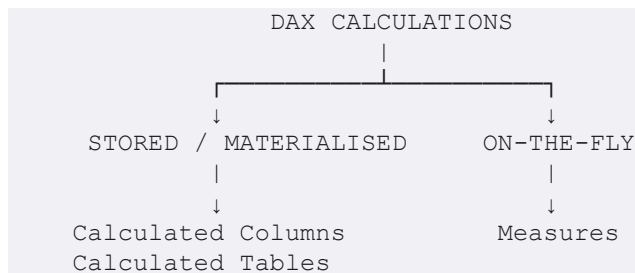
At this point your understanding should look like this:

```

DATA
↓
TABLES
↓
ROWS + COLUMNS
↓
GRAIN
↓
RELATIONSHIPS
↓
DATA MODEL
↓
BUSINESS QUESTIONS
↓
CALCULATIONS
↓
DAX

```

And calculations divide conceptually into two families:



This diagram is one of the most important diagrams in the entire course.

Model Validation Checklist

Before you trust any DAX result, check the model. Ask each question about every important table.

- Do I know what one row represents in every important table?
- Are keys unique where they should be?
- Are relationships present and correctly directed?
- Is the relationship cardinality correct?
- Do fact and dimension tables have clear roles?

- Is there a proper Date table where time analysis requires one?
- Are there unexpected duplicates?
- Are there unexpected blanks?
- Do totals reconcile with the source or business system?

The last question matters most. **Reconcile** means: compare your Power BI total with a number you trust (for example, the total in the accounting system). If they differ, the problem is usually in the data or the model, not in the DAX.

Module 2 Knowledge Check

1. What does the grain of a table mean?
2. If one row represents one sales transaction, what is the grain?
3. Which table would normally be a dimension? (A) Sales (B) Customers (C) Transactions
4. Which table would normally be a fact table? (A) Products (B) Customers (C) Sales
5. What type of relationship commonly exists between Customers and Sales?
6. Why are relationships important to DAX?
7. Why should you understand the data model before writing advanced DAX?
8. What is a primary key? What is a foreign key?
9. Why is BLANK not the same as zero?
10. Why do we need a Date table?

Answers — Module 2 Knowledge Check

1. Grain is what exactly one row of a table represents.
2. One transaction per row.
3. **B — Customers** (it describes the people involved, so it is a dimension).
4. **C — Sales** (it records the business events, so it is a fact table).
5. One-to-many: one customer to many sales.
6. Relationships define how filters travel through the model, which decides how measures respond to fields from other tables.
7. Because results depend on how the tables are connected and what each row means, even when the formula itself looks simple.
8. A primary key uniquely identifies a row in its own table. A foreign key holds values that refer to a key in another table.
9. A blank means “no value”; zero is a real number. They can look alike but mean different things, and DAX treats them differently.
10. A Date table gives a complete, reliable calendar that time-based calculations need.

Module 2 Practical Model Exercise

Assume the company has these tables:

```

Sales
- SaleID
- DateID
- CustomerID
- ProductID
- Quantity
- Sales Amount
- Cost Amount

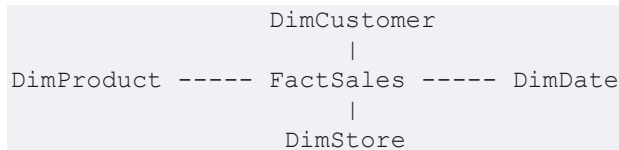
Customers
- CustomerID
- Customer Name
- Region
- Customer Type

Products
- ProductID
- Product Name
- Category
- Brand

Date
- DateID
- Date
- Year
- Month
- Quarter

```

And the model picture:



Answer the following:

1. Which table is the fact table?
2. Which tables are dimensions?
3. What is the likely grain of Sales?
4. Which columns are likely keys?
5. What are the likely one-to-many relationships?
6. Which table should provide Region?
7. Which table should provide Product Category?
8. Where would Total Sales normally be created?
9. What is one example of a row-level calculated column?
10. Why can Total Sales respond to Region even though Region is not in the Sales measure?
11. Which side is the “one” side and which is the “many” side?
12. What happens when Product Category filters FactSales?
13. Which table should contain CustomerID as a foreign key?

Answers — Module 2 Practical Model Exercise

1. **Sales** is the fact table.

2. **Customers, Products and Date** are dimensions.
3. The exact grain must be confirmed, but from the structure it is likely **one sales transaction or sales line per row**.
4. `CustomerID`, `ProductID` and `DateID` are likely **foreign keys in Sales**; `SaleID` may identify a transaction row. The same ID columns in the dimension tables are their primary keys.
5. Customers → Sales, Products → Sales and Date → Sales are commonly one-to-many relationships.
6. **Customers.**
7. **Products.**
8. As a **measure**, such as `[Total Sales]`. It is usually stored in the Sales table or in a dedicated measures table.
9. For example: `Sales Amount = Sales[Quantity] * Sales[Unit Price]`, if `Unit Price` exists in the model.
10. Because the relationship allows the Region filter from Customers to affect the relevant Sales rows.
11. The dimension tables (Customers, Products, Date) are the “one” side; Sales is the “many” side.
12. The filter travels from Products through the relationship into Sales, leaving only the sales rows for products in that category, so `[Total Sales]` adds up only those.
13. **Sales** holds `CustomerID` as a foreign key that points to Customers.

Module 2 Mini-Project

Imagine a company has the four tables above (Sales, Customers, Products, Date). Your task is to describe:

1. The fact table.
2. The dimension tables.
3. The likely relationships.
4. The grain of Sales.
5. Three possible measures.
6. One possible calculated column.
7. Which calculations are likely to be stored/materialised.
8. Which calculations should usually be measures.

Sample answer outline: (1) Sales. (2) Customers, Products, Date. (3) Customers→Sales, Products→Sales, Date→Sales, all one-to-many. (4) One sale (line) per row. (5) Total Sales, Total Cost, Profit. (6) `Sales Amount = Quantity × Unit Price`. (7) The calculated column and any calculated table. (8) Totals, profit, margin, growth and anything that must respond to slicers.

Module 2 Professional Challenge

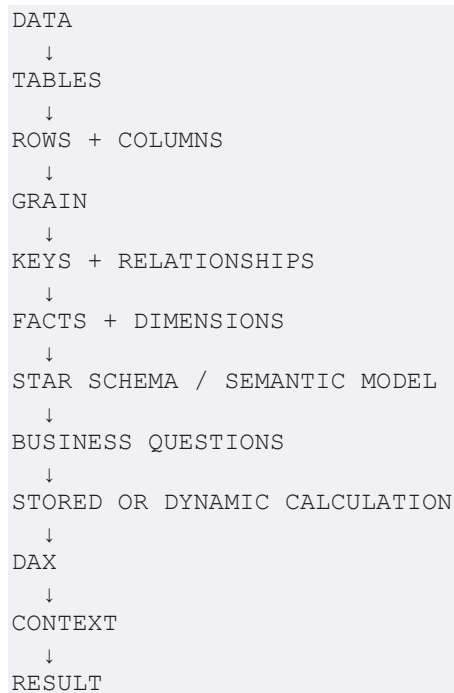
Imagine your manager asks: “**Give me total sales.**” You should not immediately write DAX. Your thinking should be:

- **Step 1:** Where is sales stored?

- **Step 2:** Is it already a column?
- **Step 3:** What does one row represent?
- **Step 4:** Do we need a row-level calculation first?
- **Step 5:** Do we need a measure?
- **Step 6:** What filters should the calculation respond to?

Only then do you write the DAX.

The Foundation Diagram



The First Major Lesson: Five Ideas to Remember

At the beginning of this course, remember these five ideas:

1. **Data** — the information.
2. **Model** — how the information is organised and connected.
3. **Persistent/Stored Calculation** — a value materialised in the model during processing/refresh.
4. **On-the-Fly/Dynamic Calculation** — a calculation evaluated when a query/report context requests it.
5. **Measure** — a reusable analytical calculation definition that is evaluated according to the current context.

If these five ideas are clear, the next parts of DAX will become much easier.

Final Foundation Checklist (Modules 1 and 2)

- I can explain what DAX is.
- I can explain what Power BI is.
- I understand the idea of a semantic model.

- I understand the difference between stored/materialised and query-time calculations.
- I know why calculated columns exist.
- I know why measures exist.
- I know what a calculated table is.
- I understand grain.
- I understand fact and dimension tables.
- I understand basic relationships.
- I understand why the model affects DAX.
- I know that a business definition comes before a DAX formula.

MODULE 3 — DAX LANGUAGE FUNDAMENTALS

Learning to read, write and understand DAX expressions

WHERE WE ARE. You already know what Power BI, DAX, semantic models, relationships, calculated columns and measures are. Now we begin learning the DAX language itself. Think of it as learning the grammar before writing essays.

Learning Objectives

- Understand what a DAX expression is.
- Understand the basic structure of a DAX formula.
- Understand functions, arguments and operators.
- Know how to reference columns, tables and measures.
- Understand the difference between a column reference and a measure reference.
- Understand constants and literals.
- Use arithmetic, comparison and logical operators.
- Understand parentheses and evaluation order.
- Write readable and professionally formatted DAX.
- Use variables with VAR and RETURN.
- Recognise common syntax errors and fix them.
- Build simple measures confidently before moving into DAX context.

What Is a DAX Expression?

An **expression** is a piece of DAX that Power BI evaluates to produce a **result**. The result might be a number, text, a date, a Boolean (TRUE/FALSE) value, a table, or a blank, depending on the expression and context.

```
2 + 3
```

The expression above produces **5**.

```
"Hello" & " World"
```

The expression above produces the text **Hello World**.

```
SUM(Sales[Sales Amount])
```

This is an **aggregation** expression (“aggregation” means combining many values into one) that produces a numeric result.

KEY IDEA. A DAX formula is not just a line of text. It is an expression that the DAX engine must interpret and evaluate. The engine reads it, works out what each piece means, and then produces a result.

A DAX Formula Has a Structure

Look at this formula:

```
Total Sales = SUM(Sales[Sales Amount])
```

Every part has a job:

Part	Meaning
Total Sales	Name of the measure
=	Assigns the expression to the measure name
SUM	DAX function
Sales[Sales Amount]	Column reference supplied to SUM

Read it aloud like a sentence: *“Total Sales is defined as the sum of the Sales Amount column in the Sales table.”*

The Left Side and Right Side

For a measure, the **left side** of the equals sign is normally the **measure name**, and the **right side** is the **DAX expression** that defines it.

```
Profit = [Total Sales] - [Total Cost]
```

The name is `Profit`. The expression is `[Total Sales] - [Total Cost]`. When you type into the formula bar in Power BI, you always type *name = expression*. If you forget the name and the equals sign, Power BI will not know what you are trying to create.

Functions

A **function** is a built-in operation provided by DAX. Functions usually have a **name** followed by **parentheses** containing **arguments**.

```
SUM(Sales[Sales Amount])
```

`SUM` is the function. `Sales[Sales Amount]` is its argument. Think of a function as a small machine: you put something in (the arguments), the machine does a fixed job, and something comes out (the result).

Function	Typical Purpose	Example
SUM	Add numeric values	<code>SUM(Sales[Sales Amount])</code>
AVERAGE	Calculate an average	<code>AVERAGE(Sales[Sales Amount])</code>
COUNTROWS	Count rows in a table	<code>COUNTROWS(Sales)</code>
DISTINCTCOUNT	Count unique values	<code>DISTINCTCOUNT(Sales[CustomerID])</code>
DIVIDE	Safely divide two expressions	<code>DIVIDE([Profit], [Total Sales])</code>

Function	Typical Purpose	Example
IF	Return one result or another	IF([Profit] > 0, "Profit", "Loss")

You do not need to memorise this list. You will meet each function again with more examples later.

Arguments

An **argument** is information supplied to a function. Some functions require one argument; others require several. Arguments are separated from each other by a **comma**.

```
DIVIDE([Profit], [Total Sales])
```

DIVIDE receives **two** arguments: the numerator (the top number) and the denominator (the bottom number).

```
IF([Profit] > 0, "Profit", "Loss")
```

IF receives **three** arguments: a condition, the result when the condition is true, and the result when it is false. Read it as: *"If Profit is greater than zero, say Profit; otherwise say Loss."*

The order of arguments matters. `DIVIDE([Profit], [Total Sales])` is profit divided by sales. Swapping them gives a completely different answer.

Parentheses

Parentheses () tell DAX which function is being called and where its arguments **begin and end**. They also control the **grouping** of expressions.

```
SUM(Sales[Sales Amount])
( [Total Sales] - [Total Cost] )
```

Do not treat parentheses as decoration. They are part of the syntax and can change the meaning of an expression. Every opening parenthesis (must have a matching closing parenthesis). A very common beginner mistake is to forget the closing one.

Tables and Columns

A column reference commonly uses the form **Table[Column]**. This tells DAX *which column in which table* you mean.

```
Sales[Quantity]
Sales[Sales Amount]
Customers[Region]
```

This is especially important when several tables contain columns with similar names. For example, both Sales and Customers might have a column called `Region`. Writing `Customers[Region]` removes all doubt.

If a table name contains spaces or special characters, it is written inside single quotes, for example `'Sales Data'[Quantity]`. To keep things simple, it is usually better to give tables short names without spaces.

Measures Are Referenced Differently

A measure is normally referenced by its **measure name**, written in **square brackets** and **without a table name**.

```
[Total Sales]
[Profit]
[Profit Margin]
```

Compare the two:

Reference	What It Refers To
Sales[Sales Amount]	A column named Sales Amount in the Sales table
[Total Sales]	A measure named Total Sales

VERY IMPORTANT. Do not confuse `Sales[Sales Amount]` with `[Total Sales]`. The first is a **column reference**. The second is a **measure reference**. Understanding this distinction is essential before learning context. A quick test: if there is a table name in front of the brackets, it is a column; if there is only a name in brackets, it is a measure.

Constants and Literals

DAX expressions can contain **literal values**: values that you type directly into the formula instead of reading from a table. They are also called **constants**.

Type	Example
Number	100
Decimal	99.50
Text	"Laptop"
Boolean	TRUE () or FALSE ()
Date	DATE (2026, 1, 1)
Blank	BLANK ()

In `IF([Profit] > 0, "Profit", "Loss")`, the `0`, `"Profit"` and `"Loss"` are all literals.

Text Values

Text values are normally enclosed in **double quotation marks**.

```
"Nigeria"
"Laptop"
```

Text can be combined (joined) with the `&` operator. Joining text is called **concatenation**.

```
Full Name = Customers[First Name] & " " & Customers[Last Name]
```

The `" "` in the middle is a text value containing a single space. Without it, `"Ada"` and `"Obi"` would be joined as `"AdaObi"`.

Arithmetic Operators

Operators are symbols that perform an operation between values.

Operator	Meaning	Example
+	Addition	10 + 5
-	Subtraction	10 - 5
*	Multiplication	10 * 5
/	Division	10 / 5
^	Exponent (power)	10 ^ 2

Examples in use:

```
Profit = [Total Sales] - [Total Cost]
Sales Amount = Sales[Quantity] * Sales[Unit Price]
```

Comparison Operators

Comparison operators compare two values. The answer is always **TRUE** or **FALSE**.

Operator	Meaning	Example
=	Equal to	[Total Sales] = 1000000
<>	Not equal to	Customers[Region] <> "South"
>	Greater than	[Profit] > 0
<	Less than	[Profit] < 0
>=	Greater than or equal to	[Total Sales] >= 1000000
<=	Less than or equal to	[Total Sales] <= 1000000

Notice that = is used in two ways. After the name it means “is defined as” (`Profit = ...`). Inside an expression it means “is equal to” (`[Region] = "South"`). The position tells you which meaning applies.

Logical Operators

Logical operators combine TRUE/FALSE conditions.

Operator	Meaning	Example
&&	AND (both must be true)	[Profit] > 0 && [Total Sales] > 100000
	OR (at least one must be true)	[Region] = "South" [Region] = "West"
NOT	Negation (flips TRUE to FALSE)	NOT([Is Active])

The `||` symbol is two vertical bars typed together. Logical operators become much more important when we begin building conditions and filters.

The DIVIDE Function

For analytical ratios, `DIVIDE` is usually preferable to manually using the division operator `/`, because it provides a clearer pattern for handling a zero or blank denominator. Dividing by zero is an error in mathematics. `DIVIDE` returns a blank (or a value you choose) instead of breaking your report.

```
Profit Margin =  
DIVIDE(  
    [Profit],  
    [Total Sales]  
)
```

Later we will study alternate-result handling with `DIVIDE` in more detail.

Operator Precedence

DAX follows rules about the **order in which operations are evaluated**. Multiplication and division generally occur before addition and subtraction.

```
10 + 5 * 2
```

This is evaluated as $10 + (5 \times 2)$, producing **20**.

```
(10 + 5) * 2
```

Parentheses force the addition to happen first, producing **30**.

PROFESSIONAL HABIT. Use parentheses when they make your intended logic clearer. Readability is often more valuable than relying on implicit precedence.

A Measure Can Use Another Measure

This is the foundation of **measure branching**.

```
Total Sales =  
SUM(Sales[Sales Amount])  
  
Total Cost =  
SUM(Sales[Cost Amount])  
  
Profit =  
[Total Sales] - [Total Cost]  
  
Profit Margin =  
DIVIDE([Profit], [Total Sales])
```

This is usually easier to read and maintain than repeating the entire Sales and Cost logic in every downstream measure. If you ever need to change how Total Sales is worked out, you change it once.

Columns vs Measures — Syntax Practice

Identify the type of each reference:

Expression	Type of Reference
Sales[Quantity]	Column
Sales[Unit Price]	Column
[Total Sales]	Measure
[Profit]	Measure
Customers[Region]	Column
Products[Category]	Column

A DAX Formula Is an Expression Tree

Professionals often think of a formula as a **structure** rather than a single line of text. For example:

```
Profit Margin = DIVIDE([Profit], [Total Sales])
```

can be drawn as a tree:

```
Profit Margin
└─ DIVIDE
   └─ [Profit]
   └─ [Total Sales]
```

The top is the final result. Each branch shows which pieces feed into it. This way of thinking becomes extremely useful when **debugging** (finding and fixing errors in) complex DAX: you can check each branch on its own.

Variables — VAR and RETURN

Variables allow you to calculate and **name** intermediate results inside a DAX expression. They can make formulas easier to read, debug and sometimes optimise.

```
Profit Margin =
VAR SalesAmount = [Total Sales]
VAR ProfitAmount = [Profit]
RETURN
    DIVIDE(ProfitAmount, SalesAmount)
```

The structure is:

```
VAR Name = Expression
RETURN
    Final Expression
```

Read the example aloud: “*Let SalesAmount be Total Sales. Let ProfitAmount be Profit. Return ProfitAmount divided by SalesAmount.*” You can use as many `VAR` lines as you need, but there must be exactly one `RETURN`, and it comes last.

Why Use Variables?

- Give **meaningful names** to intermediate calculations.
- **Avoid repeating** the same expression unnecessarily.

- Make complex logic **easier to read**.
- Make **debugging** easier: you can return a variable on its own to see what it holds.
- Create a clear **sequence of analytical steps**.

DO NOT RUSH VAR. You will use VAR heavily later. At this stage, understand the structure and the reason for using variables. We will study advanced variable behaviour later.

Comments in DAX

Comments help document logic for yourself and for other developers. Power BI ignores comments when calculating. A single-line comment starts with `//`:

```
// This calculates total sales
Total Sales = SUM(Sales[Sales Amount])
```

For longer explanations, DAX also supports **block-style comments** between `/*` and `*/`:

```
/*
    This measure calculates
    total sales.
*/
Total Sales = SUM(Sales[Sales Amount])
```

A good comment explains *why* a formula exists or what business rule it follows, not just what the formula obviously does.

Professional Naming

Names should communicate **business meaning**.

Weak Name	Better Name
Measure1	Total Sales
Calc	Profit Margin
SalesCalc	Year-to-Date Sales
ABC	Customer Count

Good naming rules:

- Use clear business terminology.
- Avoid meaningless names.
- Use consistent capitalisation.
- Do not make names unnecessarily long.
- Keep naming conventions consistent across the model.

Formatting DAX

Compare this:

```
Profit Margin = DIVIDE([Profit],[Total Sales])
```

with this:

```
Profit Margin =  
DIVIDE (  
    [Profit],  
    [Total Sales]  
)
```

Both represent the same basic logic. The second is easier to inspect and maintain, especially when expressions become complex. Good habits: put the = at the end of the first line, start the expression on a new line, put each argument on its own line, and indent with four spaces. In the Power BI formula bar, press **Shift + Enter** to start a new line, and **Tab** to indent. Power BI also offers a *Format* option in the ribbon to do this for you.

Common Syntax Errors

Problem	Typical Cause	Fix
Missing bracket	Column or measure reference incomplete	Check [] pairs
Missing parenthesis	Function call not closed	Check () pairs
Wrong separator	Arguments not separated correctly	Use the separator expected by your environment (usually a comma; some regional settings expect a semicolon)
Wrong column name	Reference does not exist	Check the model field name
Measure treated as column	Reference written incorrectly	Use [Measure Name]
Column treated as measure	Reference written incorrectly	Use Table[Column]

Read the Error, Do Not Panic

When DAX reports an error, the error message is **evidence**. Check the function name, parentheses, brackets, column names, measure names and expected argument structure **before** changing the formula randomly. Red underlines in the formula bar show where Power BI stopped understanding. Start there.

The Three Questions to Ask When a Formula Fails

1. **Is my syntax valid?** (brackets, parentheses, commas, spelling)
2. **Am I referencing the correct table, column or measure?**
3. **Does this expression make sense for the context in which I am using it?**

The third question becomes increasingly important as we enter row context and filter context.

First Complete DAX Exercise

Assume the Sales table contains:

```
Sales[Quantity]
Sales[Unit Price]
Sales[Cost]
Sales[CustomerID]
```

Create or describe the following:

1. A calculated column for Sales Amount.
2. A measure for Total Sales.
3. A measure for Total Cost, if Cost is already the total cost per row.
4. A measure for Profit.
5. A measure for Profit Margin.
6. A measure for Customer Count using DISTINCTCOUNT.

Suggested Answers

```
Sales Amount =
Sales[Quantity] * Sales[Unit Price]

Total Sales =
SUM(Sales[Sales Amount])

Total Cost =
SUM(Sales[Cost])

Profit =
[Total Sales] - [Total Cost]

Profit Margin =
DIVIDE([Profit], [Total Sales])

Customer Count =
DISTINCTCOUNT(Sales[CustomerID])
```

Notice the pattern: the first is a calculated column (row-level), and the other five are measures (report-level), and three of them reuse other measures.

Practice: Read Before You Write

For each expression, identify the **function**, the **arguments**, the **references** and the **expected type of result**.

1. SUM(Sales[Quantity])
2. AVERAGE(Sales[Unit Price])
3. DISTINCTCOUNT(Sales[CustomerID])
4. DIVIDE([Profit], [Total Sales])
5. IF([Profit] > 0, "Profit", "Loss")
6. COUNTROWS(Sales)

Answers

#	Function	Arguments	Reference type	Result type
1	SUM	Sales[Quantity]	Column	Number

#	Function	Arguments	Reference type	Result type
2	AVERAGE	Sales[Unit Price]	Column	Number (decimal)
3	DISTINCTCOUNT	Sales[CustomerID]	Column	Whole number
4	DIVIDE	[Profit] and [Total Sales]	Two measures	Number (ratio)
5	IF	condition [Profit] > 0, then "Profit", then "Loss"	One measure, two text literals	Text
6	COUNTROWS	Sales	A table	Whole number

Module 3 Assessment

1. What is a DAX expression?
2. What is a function?
3. What is an argument?
4. What is the difference between Sales[Sales Amount] and [Total Sales]?
5. What does the * operator do?
6. What does && mean?
7. What does || mean?
8. Why are parentheses important?
9. Why is DIVIDE commonly useful for ratios?
10. What do VAR and RETURN do?
11. Why are meaningful measure names important?
12. What is measure branching?
13. Why should DAX be formatted clearly?
14. What three things should you check when a formula fails?

Answers — Module 3

1. An expression is DAX logic that evaluates to a result.
2. A built-in DAX operation such as SUM, IF or DIVIDE.
3. Information supplied to a function.
4. Sales[Sales Amount] is a column reference; [Total Sales] is a measure reference.
5. Multiplication.
6. Logical AND.
7. Logical OR.
8. They define function arguments and can control grouping/evaluation order.
9. It provides a clear analytical division pattern and handles zero/blank denominator scenarios more safely than naive division.
10. VAR defines a named intermediate expression; RETURN specifies the final expression to evaluate.

11. They communicate business meaning and improve model usability and maintenance.
12. Building measures from other measures.
13. Readable formatting makes logic easier to inspect, debug and maintain.
14. Check syntax, references, and whether the logic makes sense in its evaluation context.

Module 3 Professional Challenge

Imagine a manager asks: **“Give me the average sales value for our customers.”** Do not immediately write DAX.

1. Define what “sales value” means.
2. Decide whether the average is per transaction, per order, per customer or another grain.
3. Check whether the Sales table contains the necessary data.
4. Decide whether the result should respond to report filters.
5. Choose the calculation type.
6. Then write and validate the DAX.

THE HABIT WE ARE BUILDING. Do not start with the function. Start with the business question, the grain and the intended result.

Module 3 Mental Model

```
DAX FORMULA
  ↓
Expression
  ↓
Functions + Operators + References + Values
  ↓
Evaluation
  ↓
Context
  ↓
Result
```

Final Skills Checklist (Modules 1 to 3)

- I can explain what DAX, Power BI and a semantic model are.
- I can tell a stored calculation from a measure.
- I can describe grain, keys, relationships, facts and dimensions.
- I can read a DAX formula and point to the name, function, arguments and references.
- I can tell `Table[Column]` from `[Measure]` at a glance.
- I can use arithmetic, comparison and logical operators.
- I know why DIVIDE, parentheses and variables help.
- I can format and name DAX professionally.
- I can start with the business question, not with the function.

What Comes Next?

Module 4 will focus specifically on **calculated columns**. We will examine why row-level calculations work, how Power BI evaluates a calculated column, and introduce **ROW CONTEXT** formally. After that, **Module 5** will focus deeply on **measures and FILTER CONTEXT**. Then we will study **calculated tables**, data types, aggregation, logical and text functions, and finally **context transition** and **CALCULATE**.

THE NEXT BIG IDEA. In Module 4 you will learn to think: “What is the current row?” In Module 5 you will learn to think: “What rows are currently being included in this calculation?” These two ideas lead directly into professional DAX.

PREVIEW PACK — THE NEXT MODULES AT A GLANCE

WHY THIS CHAPTER EXISTS. This chapter collects the “engine mechanics” ideas that experienced Power BI trainers teach, and puts them in simple words. You are **not** expected to master them now. Read them once so that when Modules 4 to 14 arrive, the words already feel familiar. Each idea names the module where it will be taught in full.

Lens 1 and Lens 2: Row Context and Filter Context

When DAX works out an answer, it can look at your data through **two different lenses**. Almost every confusing DAX result comes from mixing them up.

Lens	Simple meaning	Picture	Where it naturally exists
Row context	“Which single row am I standing on right now?”	A microscope that examines one row at a time	Calculated columns, and iterator functions such as SUMX
Filter context	“Which rows are currently allowed to take part in this calculation?”	A spotlight that lights up only the allowed rows	Visuals, slicers, page filters, and CALCULATE

Example of **row context**: in a calculated column `Sales Amount = Sales[Quantity] * Sales[Unit Price]`, DAX moves down the table one row at a time. On row 1 it uses row 1’s quantity and price; on row 2 it uses row 2’s, and so on.

Example of **filter context**: a visual shows `Year = 2024`. Before your measure does anything, the year slicer has already narrowed the data to 2024 rows only. The measure then calculates over that smaller set.

WHERE THIS IS TAUGHT. Module 4 (calculated columns and row context), Module 5 (measures and filter context), Modules 11 to 13 (formal row context, filter context and context transition).

Row Context Does Not Follow Relationships by Itself: RELATED and RELATEDTABLE

Here is a surprise that catches almost every beginner. Inside a calculated column you are standing on one row of one table. DAX does **not** automatically carry that row across the relationship to the other tables. If you are on a Sales row, the column `Region` that lives in the Customers table is not simply “there” for you to use. You must reach across the relationship deliberately. Two **navigation functions** do this:

Function	What it fetches	Example
<code>RELATED()</code>	One value from the “one” side of the relationship	From a Sales row, fetch that customer’s Region: <code>RELATED(Customers[Region])</code>

Function	What it fetches	Example
RELATEDTABLE ()	A whole group of rows from the “many” side	From a Customer row, fetch all Sales rows belonging to that customer: RELATEDTABLE (Sales)

Sales row — RELATED() → one Customers value (many → one)
 Customer row — RELATEDTABLE() → many Sales rows (one → many)

An easy way to remember it: **RELATED** goes toward the “one” side and brings back one value.
RELATEDTABLE goes toward the “many” side and brings back a table.

A CAREFUL CORRECTION. Some training material says these functions “reactivate” or “wire back” a deactivated relationship. That is not accurate. They use the relationship that already exists in the model. If the relationship is missing, or if it is an inactive one, they will not work the way you expect. Inactive relationships are switched on with USERELATIONSHIP () inside CALCULATE, which is a different tool.

RULE OF THUMB. When you are inside a calculated column and need information that lives in another table, ask: “Do I need one value (RELATED) or a group of rows (RELATEDTABLE)?”

Iterators: Why SUMX Has an “X”

Two kinds of functions add numbers up.

Type	Example	What it does
Aggregator	SUM (Sales[Sales Amount])	Takes one existing column and adds it up.
Iterator (the “X” functions)	SUMX (Sales, Sales[Quantity] * Sales[Unit Price])	Walks through a table one row at a time , does a calculation on each row, stores each row’s answer, and then adds the answers together.

Look at this example. Three sales rows, each with Price 100 and Quantity 5:

Row	Price	Quantity	Price × Quantity
1	100	5	500
2	100	5	500
3	100	5	500
		SUMX result	1,500

SUMX multiplied row by row (500 each) and then added: **1,500**. If you had no Sales Amount column, you could not use plain SUM, because the amount is not stored anywhere. SUMX creates it row by row in memory.

The shape of every iterator is the same:

```
SUMX( <table>, <expression> )
```

First tell the function **which table to walk through**, then give it **the calculation to do on each row**. Other X functions follow the same pattern: AVERAGEX, MINX, MAXX, COUNTX and RANKX.

A very common beginner mistake is to try `SUM(Price) * SUM(Quantity)`. That adds all prices (300) and all quantities (15), then multiplies: 4,500. This is **wrong**, because it multiplies two totals instead of multiplying each row first.

WHERE THIS IS TAUGHT. Module 8 (aggregation functions) and the iterators module.

CALCULATE: The Function That Changes the Filter Context

Remember that a measure always starts inside the filter context of the report. **CALCULATE** is the function that lets you change that filter context on purpose. Picture a funnel of data coming down from the report, and CALCULATE as a wrench on the pipe that lets you open or close valves.

Example. A slicer says `Country = Australia`. You want a measure that ignores the country slicer and always shows 2024:

```
Sales 2024 All Countries =  
CALCULATE(  
    [Total Sales],  
    ALL(Customers[Country]),  
    'Date'[Year] = 2024  
)
```

- `ALL(Customers[Country])` **removes** the existing country filter (the Australia slicer no longer applies).
- `'Date'[Year] = 2024` **adds** a brand-new filter.
- `[Total Sales]` is then calculated in this **modified** filter context.

So CALCULATE can do three things that ordinary measures cannot do easily: **ignore** existing report filters, **replace** them, and **inject new business rules** into a calculation.

A CAREFUL CORRECTION. You may hear that CALCULATE is “the only function” that can change filter context or turn row context into filter context. That is too strong. CALCULATE and its table-returning sister **CALCULATETABLE** are the main tools for changing filter context. Also, when you use a **measure** inside a row context (for example inside an iterator), DAX automatically wraps that measure in a CALCULATE, which causes the same row-to-filter conversion. That behaviour is called **context transition**, and you will study it in Module 13. For beginners, “CALCULATE is the main way to change filter context” is the safe sentence.

Time Intelligence: Comparing With the Past

Business leaders constantly ask: “How are we doing compared with last year?” **Time intelligence** functions answer that by shifting the dates the calculation looks at.

```
Prior Year Sales =  
CALCULATE(  
    [Total Sales],  
    DATEADD('Date'[Date], -1, YEAR)  
)
```

DATEADD is the “time-shift engine”. It moves the date filter back by one year, so the same measure now reads last year’s rows. Put `[Total Sales]` and `[Prior Year Sales]` on a line chart by month and you can compare them directly.

THE GOLDEN REQUIREMENT. Time intelligence breaks if the Date table has gaps. The Date table must hold **every single day**, in one unbroken run covering whole years (1 January to 31 December), and it must be connected to the fact table with a **one-to-many** relationship. Also mark it as the model’s Date table. One missing day can quietly give wrong year-to-date and previous-year numbers.

Visual Calculations: Calculating on What the Visual Already Shows

A newer Power BI feature called **visual calculations** lets you write DAX that works on the **data inside a visual** instead of on the whole model. Examples are `RUNNINGSUM()` for a running total and `PREVIOUS()` for the value in the previous row of the visual.

```
Where each kind of calculation lives
-----
Physical data model tables    ← calculated columns / tables write here
DAX measures                  ← read the model, produce numbers for the
visual
Presentation layer (visual) ← visual calculations read and write here
```

They are handy for “versus previous” logic (for example, this month against the previous month in the same table or chart) without adding complicated measures to the model.

A CAREFUL CORRECTION. Some summaries say visual calculations “read from the pixels on the screen”. They actually read the **data values of that one visual** (the rows and columns it is displaying), not images. Because the feature is relatively new, menu names and availability can differ between Power BI versions, so check what your version offers.

DAX Query View and Copilot

DAX Query View is a workspace inside Power BI Desktop where you write and run DAX **queries** to look at results as a table, test a measure on its own, and study how measures depend on other measures. Professionals use it to debug a calculation without disturbing the live report. For example, if `[Profit Margin]` is built from `[Total Sales]` and `[Total Cost]`, tracing those links shows exactly where a number comes from.

Copilot can draft DAX from a plain-English request such as “Create a measure for period-over-period sales”. It can save time. But:

AI WARNING. Copilot is an **accelerator, not an architect**. It cannot repair a weak data model. Every AI-written measure must be tested in several contexts (a single day, a month, a year, with and without slicers) to make sure the totals are right. Never publish a measure you cannot explain yourself.

The Complete Anatomy of a Professional Power BI Workflow

Putting everything together, a professional solution moves left to right through five stages. Each stage has a preferred tool and a preferred way of thinking.

Stage	Name	Tools and ideas
1	Extract / Transform	SQL or Power Query for the static, heavy lifting
2	Model	Clean star schemas with one-to-many relationships in a single direction
3	Row context	<code>RELATED ()</code> to bridge tables; <code>SUMX ()</code> to iterate row by row
4	Filter context	<code>CALCULATE ()</code> to change filters and inject business rules
5	Presentation	Visual calculations, data bars and clear visuals for quick insight

FINAL TAKEAWAY. Mastery of Power BI is not memorising 200 DAX formulas. It is mastering **context**, managing the **flow of data**, and understanding the invisible mechanics of the engine underneath, which is called **VertiPaq**. You will study VertiPaq, the Storage Engine and the Formula Engine in the performance modules.

Preview Pack Knowledge Check

1. Which context asks “which single row am I on?” and which asks “which rows are allowed in?”
2. You are in a calculated column on the Sales table and need the customer’s Region from the Customers table. Which function do you use?
3. What does `RELATEDTABLE` return?
4. Why is `SUM(Price) * SUM(Quantity)` wrong for revenue?
5. In `SUMX(Sales, Sales[Quantity] * Sales[Unit Price])`, what are the two arguments?
6. In the `CALCULATE` example, what does `ALL(Customers[Country])` do?
7. Name the golden requirement for a Date table.
8. Why must AI-generated DAX always be tested?

Answers — Preview Pack

1. **Row context** asks “which row am I on?”. **Filter context** asks “which rows are allowed to take part?”.
2. `RELATED(Customers[Region])`, because you need one value from the “one” side.
3. A table: all the rows from the “many” side that belong to the current row.
4. It multiplies two **totals**. The correct method multiplies each row first, then adds (`SUMX`).
5. The table to walk through (`Sales`) and the expression to calculate on every row.
6. It removes the existing country filter so the measure ignores the Country slicer.
7. It must contain every day with no gaps, in whole years, with a one-to-many relationship to the fact table (and marked as the Date table).
8. Because AI can write plausible but wrong logic, and it cannot fix a weak model. Testing in different contexts shows whether totals stay correct.

APPENDIX — QUICK-REVIEW FLASHCARDS

Cover the right-hand column, read the question, answer out loud, then check. Come back to the cards you miss.

Question	Answer
What does DAX stand for?	Data Analysis Expressions.
What is a semantic model?	The organised analytical structure of tables, columns, relationships and measures that DAX works inside.
What does “grain” mean?	Exactly what one row of a table represents.
What is a primary key?	A column that uniquely identifies each row in its own table.
What is a foreign key?	A column whose values refer to the key of another table.
What is a fact table?	A table of business events (sales, orders) holding the numbers you add up.
What is a dimension table?	A table describing the who, what, when and where (customers, products, dates).
What is a star schema?	One central fact table surrounded by dimension tables.
Which way do filters normally flow?	From the “one” side (dimension) to the “many” side (fact).
Why avoid bidirectional filters by default?	They can create several filter paths and ambiguous results.
Stored or dynamic: calculated column?	Stored. Calculated at refresh and kept in the model.
Stored or dynamic: measure?	Dynamic. Evaluated for the current query context.
When do calculated column values change?	At refresh time, when the model is processed.
When do measure results change?	At query time, whenever the report context changes.
Does “persistent” mean “permanent”?	No. A refresh can recalculate stored values.
What is an implicit measure?	One Power BI creates automatically when you drag a numeric column into a visual.
Why write explicit measures?	Control, naming, formatting, reuse and one place to change a definition.
What is the upstream rule?	Do the work as far left as possible: database, then Power Query, then DAX columns, then measures.

Question	Answer
What is query folding?	Power Query turning your steps into a query that the source database runs.
What are the four homes of DAX?	Calculated columns/tables, measures, visual calculations, and the impossible zone (visual to stored model).
Which is a column reference and which a measure reference?	<code>Sales[Quantity]</code> is a column; <code>[Total Sales]</code> is a measure.
What does DIVIDE do better than <code>/</code> ?	It handles a zero or blank denominator safely.
What do VAR and RETURN do?	VAR names an intermediate result; RETURN gives the final answer.
What is measure branching?	Building new measures out of existing measures.
What is row context?	“Which single row am I on?”
What is filter context?	“Which rows are currently allowed into the calculation?”
What does RELATED return?	One value from the “one” side of a relationship.
What does RELATEDTABLE return?	A table of matching rows from the “many” side.
How does SUMX differ from SUM?	SUMX calculates row by row first, then adds; SUM only adds an existing column.
What is the main job of CALCULATE?	To change the filter context a calculation runs in.
What must a good Date table be?	Complete (no missing days), in whole years, related one-to-many, and marked as a Date table.
What is the safe rule for AI-written DAX?	Test it in several contexts and make sure you can explain it.

Power BI DAX — From Complete Beginner to Professional